

2026 年 6 月 4 日

PostgreSQL 19 新機能検証結果 (Beta 1)

篠田典良

目次

目次.....	2
1. 本文書について.....	5
1.1. 本文書の概要.....	5
1.2. 本文書の対象読者	5
1.3. 本文書の範囲.....	5
1.4. 本文書の対応バージョン	5
1.5. 本文書に対する質問・意見および責任	6
1.6. 表記	6
2. PostgreSQL 19 における変更点概要	8
2.1. 大規模環境に対応する新機能	8
2.2. 信頼性向上に関する新機能.....	8
2.3. 運用性向上に関する新機能.....	9
2.4. プログラミングに関する新機能	9
2.5. 非互換	10
2.5.1. サポート情報.....	10
2.5.2. トランザクション周回の警告	10
2.5.3. LSN の表記	11
2.5.4. LDAP プロトコル	11
2.5.5. オブジェクト名	11
2.5.6. GUC	11
2.5.7. MD5 認証	12
2.5.8. pg_statistic_ext_data カタログ	13
2.5.9. ON CONFLICT 句	14
2.5.10. RADIUS 認証	14
2.5.11. psql コマンド	14
2.5.12. MULE_INTERNAL エンコーディング	15
2.5.13. COPY FROM 文	15
2.5.14. JSON_ARRAY 関数.....	15
3. 新機能解説.....	17
3.1. アーキテクチャの変更	17
3.1.1. システムカタログの変更.....	17
3.1.2. ロジカル・レプリケーションの拡張	25
3.1.3. パーティション・テーブルの拡張	30
3.1.4. SQL Property Graph	33

3.1.5. SSL/SNI.....	34
3.1.6. チェックサムの動的変更.....	35
3.1.7. ロール.....	36
3.1.8. Libpq.....	37
3.1.9. I/O 処理.....	38
3.1.10. 自動 VACUUM.....	39
3.1.11. フック.....	41
3.1.12. 高速化.....	41
3.1.13. ログ出力.....	42
3.1.14. OAuth 認証.....	43
3.1.15. 待機イベント.....	43
3.2. SQL 文の拡張.....	44
3.2.1. ALTER DEFAULT PRIVILEGE 文.....	44
3.2.2. ALTER DOMAIN 文.....	44
3.2.3. ALTER FOREIGN DATA WRAPPER 文.....	44
3.2.4. ALTER PUBLICATION 文.....	45
3.2.5. ALTER SUBSCRIPTION 文.....	45
3.2.6. ALTER TABLE 文.....	46
3.2.7. CHECKPOINT 文.....	48
3.2.8. COPY 文.....	49
3.2.9. CREATE PUBLICATION 文.....	52
3.2.10. CREATE SCHEMA 文.....	53
3.2.11. CREATE STATISTICS 文.....	54
3.2.12. DELETE / UPDATE 文.....	55
3.2.13. EXPLAIN 文.....	56
3.2.14. GRANT 文.....	59
3.2.15. INSERT 文.....	60
3.2.16. REPACK 文.....	61
3.2.17. SELECT 文.....	62
3.2.18. SET 文.....	64
3.2.19. VACUUM 文.....	65
3.2.20. WAIT FOR LSN 文.....	66
3.2.21. 関数.....	68
3.2.22. JSON.....	76
3.2.23. データ型.....	78
3.2.24. PL/pgSQL.....	80

3.2.25. PL/Python.....	81
3.2.26. オプティマイザー.....	81
3.3. パラメーターの変更.....	90
3.3.1. 追加されたパラメーター.....	90
3.3.2. 変更されたパラメーター.....	92
3.3.3. デフォルト値が変更されたパラメーター.....	93
3.3.4. 削除されたパラメーター.....	94
3.4. ユーティリティ.....	95
3.4.1. oid2name.....	95
3.4.2. pg_archivecleanup.....	95
3.4.3. pg_createsubscriber.....	95
3.4.4. pg_dump.....	97
3.4.5. pg_dumpall.....	98
3.4.6. pg_restore.....	99
3.4.7. pg_upgrade.....	99
3.4.8. pg_verifybackup.....	100
3.4.9. pg_waldump.....	100
3.4.10. psql.....	100
3.4.11. vacuumdb.....	104
3.5. Contrib モジュール.....	106
3.5.1. auto_explain.....	106
3.5.2. basic_archive.....	107
3.5.3. file_fdw.....	107
3.5.4. injection_points.....	108
3.5.5. pg_buffercache.....	109
3.5.6. pg_plan_advice.....	110
3.5.7. pg_stash_advice.....	112
3.5.8. pg_stat_statements.....	115
3.5.9. postgres_fdw.....	117
3.5.10. その他.....	119
参考にした URL.....	120
変更履歴.....	121

1. 本文書について

1.1. 本文書の概要

本文書はオープンソース RDBMS である PostgreSQL 19 (19.0) Beta 1 の主な新機能について検証した文書です。

1.2. 本文書の対象読者

本文書は、既にある程度 PostgreSQL に関する知識を持っているエンジニア向けに記述しています。インストール、基本的な管理等は実施できることを前提としています。

1.3. 本文書の範囲

本文書は PostgreSQL 18 (18.4) と PostgreSQL 19 Beta 1 (19.0) の主な差分を記載しています。原則として利用者が見て変化がわかる機能について調査しています。すべての新機能について記載および検証しているわけではありません。特に以下の新機能は含みません。

- バグ解消
- 内部動作の変更によるパフォーマンス向上
- レグレッション・テストの改善
- psql コマンドのタブ入力による操作性改善
- pgbench コマンドの改善
- ドキュメントの改善、ソース内の Typo 修正
- 動作に変更がないリファクタリング

1.4. 本文書の対応バージョン

本文書は以下のバージョンとプラットフォームを対象として検証を行っています。

表 1 対象バージョン

種別	バージョン
データベース製品	PostgreSQL 18.4 (比較対象)
	PostgreSQL 19 Beta 1 (19.0) (2026/06/01 20:05:06)
オペレーティング・システム	Red Hat Enterprise Linux 9 Update 5 (x86-64)
Configure オプション	--with-ssl=openssl --with-lz4 --with-zstd --with-llvm --with-libxml --enable-injection-points --with-libnuma --with-liburing

1.5. 本文書に対する質問・意見および責任

本文書の内容は PostgreSQL Global Development Group の公式見解ではありません。また内容の間違いにより生じた問題について作成者および所属企業は責任を負いません。本文書で検証した仕様は後日予告なく変更される場合があります。本文書に対するご意見等ありましたら作成者 篠田典良 (Mail: noriyoshi.shinoda@gmail.com) までお知らせください。

1.6. 表記

本文書内にはコマンドや SQL 文の実行例および構文の説明が含まれます。実行例は以下のルールで記載しています。

表 2 例の表記ルール

表記	説明
#	Linux root ユーザーのプロンプト
\$	Linux 一般ユーザーのプロンプト
太字	ユーザーが入力する文字列
postgres=#	SUPERUSER 属性を持つ PostgreSQL ユーザーが利用する psql コマンド・プロンプト
postgres=>	SUPERUSER 属性を持たない PostgreSQL ユーザーが利用する psql コマンド・プロンプト
<u>下線部</u>	特に注目すべき項目
...	より多くの情報が出力されるが文書内では省略していることを示す
<<パスワード>>	パスワードの入力を示す

構文は以下のルールで記載しています。

表 3 構文の表記ルール

表記	説明
<i>斜体</i>	ユーザーが利用するオブジェクトの名前やその他の構文に置換
[]	省略できる構文であることを示す
{ A B }	A または B を選択できることを示す
...	旧バージョンと同一である一般的な構文

2. PostgreSQL 19 における変更点概要

PostgreSQL 19 には 200 以上の新機能が追加されました。本章では代表的な新機能と利点の概要について説明します。新機能の詳細は「3. 新機能解説」で説明します。

2.1. 大規模環境に対応する新機能

大規模環境に適用できる以下の機能が追加されました。

- パーティションの分割とマージ

パーティション・テーブルに対して複数のパーティションのマージと、単一パーティションの分割機能が実装されました。

- テーブル設定の動的変更

仮想列に対する計算方式や、CHECK 制約に対する ENFORCED 設定が ALTER TABLE 文で行えるようになりました。

- 自動 VACUUM でパラレル・ワーカー使用

従来のバージョンでは自動 VACUUM ではパラレル・ワーカーは使用されませんでした。PostgreSQL 19 では自動 VACUUM も並列処理が行われるようになりました。

2.2. 信頼性向上に関する新機能

信頼性を向上させるために以下の拡張が実装されました。

- pg_dumpall コマンドの出力フォーマット

pg_dumpall コマンドにテキスト以外の出力フォーマットを指定できるようになりました。テキスト以外を選択した場合、リストアは pg_restore コマンドを使用します。

- SSL/SNI 対応

PostgreSQL サーバー側で SSL/SNI に対応しました。SNI 設定を行うための pg_hosts.conf ファイルが利用できるようになりました。

2.3. 運用性向上に関する新機能

運用性を向上できる以下の機能が追加されました。

□ シーケンスのレプリケーション

ロジカル・レプリケーションでシーケンスのレプリケーションがサポートされます。またパブリケーション側でレプリケーション対象外テーブルの指定を行えるようになりました。

□ REPACK 文

VACUUM FULL 文と CLUSTER 文の処理を同時に行う REPACK 文が追加されました。

2.4. プログラミングに関する新機能

SQL 文に以下の機能が追加されました。

□ SQL Property Graph

グラフ問合せを行うオブジェクト PROPERTY GRAPH が作成できるようになりました。PROPERTY GRAPH 検索のための GRAPH_TABLE 関数が提供されています。

□ INSERT 文の拡張

INSERT ON CONFLICT 文に DO SELECT 句を指定できるようになりました。

□ UPDATE / DELETE 文の拡張

UPDATE 文と DELETE 文に範囲型の部分更新を行う ON PORTION OF 句がサポートされます。

□ GROUP BY ALL 句

すべて出力列で GROUP BY 処理を行う GROUP BY ALL 句がサポートされます。

□ 新規 Contrib モジュール

実行計画を制御する Contrib モジュール pg_plan_advice、pg_stash_advice が追加されました。

2.5. 非互換

PostgreSQL 19 は PostgreSQL 18 から以下の仕様が変更されました。

2.5.1. サポート情報

PostgreSQL 19 では以下のプラットフォームやツール向けのサポート・バージョンが変更されました。

PostgreSQL 19 のビルドに必要なコンポーネントのサポート・バージョンの変化は以下の通りです。

- Meson 0.57.2 以上 [[f039c22](#)]
- gcc 4.7 以上 [[4fbe015](#)]
- C11 規格 [[f5e0186](#)]
- Visual Studio 2019 以降 [[8fd9bb1](#)]

PostgreSQL 19 では以下のコンポーネントがサポートされるようになりました。

- Tcl 9 のサポート追加 [[32a2aa7](#)]

PostgreSQL 19 では以下の OS のサポートが復活しました。

- IBM AIX [[4a1b05c](#)]

PostgreSQL 19 では以下の規格がサポートされるようになりました。

- UNICODE 17.0.0 [[57ee397](#)]
- GB18030 2022 [[5334620](#)]
- C23 規格 [[76f4b92](#)]

2.5.2. トランザクション周回の警告

トランザクション ID 周回の警告メッセージに残りパーセンテージを示す **DETAIL** ログが追加されました。また警告が出力される残 ID 数が 4,000 万から 1 億に変更されました。
[[e646450](#), [48f11bf](#)]

例 1 トランザクション周回警告ログメッセージ

```
WARNING: database "postgres" must be vacuumed within 3001666 transactions
DETAIL:  Approximately 0.14% of transaction IDs are available for use.
HINT:   To avoid transaction ID assignment failures, execute a database-wide
VACUUM in that database.
You might also need to commit or roll back old prepared transactions, or drop
stale replication slots.
```

2.5.3. LSN の表記

LSN の表記が統一され、下位 32 ビット部分が常にゼロ・パディングされて表示されるようになりました。一部のビューや関数の戻り値の表記に互換性がありません。[[2633dae](#)]

例 2 LSN の表記

```
postgres=# SELECT * FROM
              pg_create_logical_replication_slot('slot1', 'pgoutput') ;
 slot_name |      lsn
-----+-----
 slot1     | 0/019D2F30
(1 row)
```

2.5.4. LDAP プロトコル

LDAP サーバーとの接続に LDAP プロトコル・バージョン 3 を使用するようになりました。従来はバージョン 2 を使っていました。[[5f19d13](#)]

2.5.5. オブジェクト名

データベース、ロール、テーブル空間の名前に CR コード、LF コードが使用できなくなりました。[[b380a56](#)]

2.5.6. GUC

PostgreSQL 19 ではパラメーター（GUC）の設定について以下の非互換があります。

□ `standard_conforming_strings` パラメーター

このパラメーターは `off` に設定変更できなくなりました。[[4576208](#)]

例 3 `standard_conforming_strings` パラメーターの変更

```
postgres=> SET standard_conforming_strings = off ;
ERROR: non-standard string literals are not supported
```

□ パラメーターのリセット

`ALTER DATABASE RESET` 文と `ALTER USER RESET` 文に存在しないパラメーターを指定するとエラーが発生します。従来はエラーになりませんでした。[[9a71989](#)]

例 4 `ALTER DATABASE/USER RESET` 文の実行

```
postgres=# ALTER DATABASE postgres RESET foo ;
ERROR: unrecognized configuration parameter "foo"
postgres=# ALTER USER postgres RESET foo ;
ERROR: unrecognized configuration parameter "foo"
```

2.5.7. MD5 認証

パラメーター `md5_password_warnings` が `on` (デフォルト値) の場合、MD5 パスワードによる認証時に警告が出力されるようになりました。[[bc60ee8](#)]

例 5 MD5 認証時の警告

```
$ psql -d postgres -U user1 -h dbvr1
Password for user user1: <<PASSWORD>>
WARNING: authenticated with an MD5-encrypted password
DETAIL: MD5 password support is deprecated and will be removed in a future
release of PostgreSQL.
psql (19beta1)
Type "help" for help.
```

2.5.8. pg_statistic_ext_data カタログ

pg_statistic_ext_data カタログの stxnddistinct 列および stxddependencies 列の JSON 出力フォーマットが変更されました。[1f927cc, e76defb]

例 6 PostgreSQL 18 の出力フォーマット

```
postgres=# SELECT jsonb_pretty(d.stxnddistinct::text::jsonb) AS stxnddistinct,
                  jsonb_pretty(d.stxddependencies::text::jsonb) AS stxddependencies
FROM pg_statistic_ext s LEFT JOIN pg_statistic_ext_data d ON (d.stxoid = s.oid)
WHERE s.stxname = 'data1_col1_col2' ;
 stxnddistinct | stxddependencies
-----+-----
{              +| {              +
  "1, 2": 10+|  "1 => 2": 1.000000+
}              | }
(1 row)
```

例 7 PostgreSQL 19 の出力フォーマット

```
postgres=# SELECT jsonb_pretty(d.stxnddistinct::text::jsonb) AS stxnddistinct,
                  jsonb_pretty(d.stxddependencies::text::jsonb) AS stxddependencies
FROM pg_statistic_ext s LEFT JOIN pg_statistic_ext_data d ON (d.stxoid = s.oid)
WHERE s.stxname = 'data1_col1_col2' ;
 stxnddistinct | stxddependencies
-----+-----
[              +| [              +
  {              +| {              +
    "ndistinct": 10,+|    "degree": 1.000000,+
    "attributes": [ +|    "attributes": [  +
      1,+|      1              +
      2  +|    ],              +
    ]    +|    "dependency": 2  +
  }      +|  }              +
]        | ]
(1 row)
```

2.5.9. ON CONFLICT 句

PostgreSQL 19 では INSERT ON CONFLICT 文でオペレーター・クラスを指定するとエラーが発生します。下記の INSERT 文は PostgreSQL 18 では成功していました。
[630a937]

例 8 ON CONFLICT 句とオペレーター・クラスの指定

```
postgres=> INSERT INTO data1 VALUES (1, 'data1') ON CONFLICT
            (col1 INT4_OPS (fillfactor=10)) DO NOTHING ;
ERROR:  operator class options are not allowed in ON CONFLICT clause
LINE 1: INSERT INTO data1 VALUES (1, 'data1') ON CONFLICT (col1 INT4_OPS (fi...
                                     ^
```

2.5.10. RADIUS 認証

RADIUS/UDP 認証がサポートされなくなりました。[a1643d4]

2.5.11. psql コマンド

パーティション・テーブルと、継承テーブルに対する¥d+メタコマンドの出力が変更されました。[7bff9f1, 1a11405]

例 9 PostgreSQL 18 の出力フォーマット

```
postgres=> CREATE TABLE part1(col1 INT, col2 INT) PARTITION BY LIST (col1) ;
CREATE TABLE
postgres=> CREATE TABLE part1v1 PARTITION OF part1 FOR VALUES IN (100) ;
CREATE TABLE
postgres=> ¥d+ part1

                                Partitioned table "public.part1"
 Column | Type   | Collation | Nullable | Default | Storage | Compress...
-----+-----+-----+-----+-----+-----+-----
 col1   | integer |           |          |         | plain   |          ...
 col2   | integer |           |          |         | plain   |          ...
Partition key: LIST (col1)
Partitions: part1v1 FOR VALUES IN (100)
```

例 10 PostgreSQL 19 の出力フォーマット

```
postgres=> CREATE TABLE part1(col1 INT, col2 INT) PARTITION BY LIST (col1) ;
CREATE TABLE
postgres=> CREATE TABLE part1v1 PARTITION OF part1 FOR VALUES IN (100) ;
CREATE TABLE
postgres=> \d+ part1
```

Partitioned table "public.part1"

Column	Type	Collation	Nullable	Default	Storage	Compress...
col1	integer				plain	...
col2	integer				plain	...

Partition key: LIST (col1)

Partitions:

part1v1 FOR VALUES IN (100)

2.5.12. MULE_INTERNAL エンコーディング

文字エンコード MULT_INTERNAL は削除されました。これに伴い pg_upgrade コマンドはサポートされないエンコーディングをチェックするようになりました。[77645d4]

2.5.13. COPY FROM 文

WHERE 句にシステム列を用いる COPY FROM 文は実行できなくなりました。[21c69dc]

例 11 システム列を用いる WHERE 句

```
postgres=# COPY data1 FROM stdin WHERE xmin = 100 ;
ERROR: system columns are not supported in COPY FROM WHERE conditions
DETAIL: Column "xmin" is a system column.
```

2.5.14. JSON_ARRAY 関数

JSON_ARRAY 関数に空の結果セットが指定された場合、PostgreSQL 18 までの JSON_ARRAY 関数は NULL を返していました。PostgreSQL 19 の JSON_ARRAY 関数は SQL/JSON 標準に合わせて空の JSON 配列が返ります。[8d829f5]

例 12 PostgreSQL 18 の出力

```
postgres=> ¥set null NULL
Null display is "NULL".
postgres=> SELECT JSON_ARRAY(SELECT i FROM (VALUES (1), (2), (NULL), (4)) foo(i)
WHERE i > 4) ;
  json_array
-----
      NULL
(1 row)
```

例 13 PostgreSQL 19 の出力

```
postgres=> SELECT JSON_ARRAY(SELECT i FROM (VALUES (1), (2), (NULL), (4)) foo(i)
WHERE i > 4) ;
  json_array
-----
      []
(1 row)
```


3. 新機能解説

3.1. アーキテクチャの変更

PostgreSQL 18 で拡張されたアーキテクチャや機能について解説しています。

3.1.1. システムカタログの変更

以下のシステムカタログやビューが変更されました。[[228c370](#), [167ed80](#), [fd5a1a0](#), [deb6744](#), [b71bae4](#), [a5b5432](#), [a850be2](#), [185e304](#), [d3b6183](#), [96b3784](#), [f9a09aa](#), [f6a4c49](#), [3edaf29](#), [76b7872](#), [e68b6ad](#), [5db6a34](#), [ab40db3](#), [0d78952](#), [f3c9e34](#), [c257ba8](#), [a36164e](#), [307447e](#), [fb80f38](#), [ac58465](#), [723619e](#), [8fe315f](#), [2f094e7](#), [3b88e50](#), [4019f72](#), [01d485b](#), [87f61f0](#)]

表 4 追加されたシステムカタログ／ビュー

カタログ／ビュー名	説明
pg_dsm_registry_allocations	動的共有メモリー（DSM）の情報を提供
pg_propgraph_element	Property Graph エlement
pg_propgraph_element_label	Property Graph エlement・ラベル
pg_propgraph_label	Property Graph ラベル
pg_propgraph_label_property	Property Graph ラベル・プロパティ
pg_propgraph_property	Property Graph プロパティ
pg_publication_sequences	PUBLICATION に含まれるシーケンス
pg_stat_autovacuum_scores	テーブル毎の VACUUM スコア情報
pg_stat_lock	ロック状態の実行統計
pg_stat_progress_data_checksums	チェックサム設定変更の進捗情報レポート
pg_stat_progress_repack	REPACK 文の進捗状況レポート
pg_stat_recovery	起動プロセスのリカバリー状況をレポート

表 5 列が追加されたシステムカタログ／ビュー

カタログ／ビュー名	追加列名	データ型	説明
pg_available_extensions	location	text	モジュールの格納ディレクトリ
pg_available_extensions_versions	location	text	モジュールの格納ディレクトリ

カタログ／ビュー名	追加列名	データ型	説明
pg_foreign_data_wrapper	fdwconnection	oid	Foreign Data Wrapper 接続関数 OID
pg_publication	puballsequences	boolean	FOR ALL SEQUENCES が指定されているか
pg_publication_rel	prexcept	boolean	レプリケーションの例外テーブルかを示す
pg_range	rngconstruct2	regproc	2 引数のコンストラクタ OID
	rngconstruct3	regproc	3 引数のコンストラクタ OID
	rngmltconstruct0	regproc	0 引数のマルチ範囲コンストラクタ OID
	rngmltconstruct1	regproc	1 引数のマルチ範囲コンストラクタ OID
	rngmltconstruct2	regproc	2 引数のマルチ範囲コンストラクタ OID
pg_replication_slots	slotsync_skip_reason	text	スロット同期をスキップした理由
pg_stats	tableid	oid	テーブルの OID
pg_stats_ext	attnum	smallint	列の数
	tableid	oid	テーブルの OID
	statistics_id	oid	拡張統計オブジェクト OID
pg_stats_ext_exprs	tableid	oid	テーブルの OID
	statistics_id	oid	拡張統計オブジェクト OID
	range_length_histogram	anyarray	空でない範囲値長のヒストグラム
	range_empty_frac	real	空の範囲値の割合
	range_bounds_histogram	anyarray	範囲値の上限・下限のヒストグラム
pg_stat_database_conflicts	stats_reset	timestampz	データ・リセット時刻
pg_stat_progress_analyze	started_by	text	ANALYZE 実行契機 (manual, autovacuum)

カタログ／ビュー名	追加列名	データ型	説明
pg_stat_progress_basebackup	backup_type	text	バックアップ方法の種類を示す (full または incremental)
pg_stat_progress_vacuum	mode	text	VACUUM 実行モード (normal, aggressive, failsafe)
	started_by	text	VACUUM 実行契機 (manual, autovacuum, autovacuum_wraparound)
pg_stat_replication_slots	mem_exceeded_count	bigint	デコード・メモリー量が logical_decoding_work_memory を超えた回数
	slotsync_skip_count	bigint	レプリケーションスロットの同期スキップ回数
	slotsync_last_skip	timestampz	レプリケーションスロットの同期スキップ最終日時
pg_stat_subscription_stats	confl_update_deleted	bigint	更新対象が他のソースで削除された回数
	sync_seq_error_count	bigint	シーケンスの同期エラー数
	sync_table_error_count	bigint	テーブルの同期エラー数
pg_stat_user_functions	stats_reset	timestampz	統計情報リセット時刻
pg_stat_wal	wal_fpi_bytes	numeric	ページ全体書き込み (Full Page Write) バイト数
pg_subscription	subretained_tuples	boolean	update_deleted の検出が有効か
	submaxretention	int	デッドタブルの最大保持期間
	subretention_active	boolean	デッドタブルを保持するか
	subserver	oid	PUBLICATION 接続用 FOREIGN SERVER の OID
	subwalrcvtimeout	text	WAL receiver のタイムアウト

カタログ／ビュー名	追加列名	データ型	説明
pg_stat_{all sys user}_{tables indexes}	stats_reset	timestamp z	統計情報リセット時刻
pg_statio_{all sys user}_{tables indexes}	stats_reset	timestamp z	統計情報リセット時刻

表 6 列が削除された pg_catalog スキーマ内のビュー

カタログ／ビュー名	削除列名	説明
pg_stat_subscription_stats	sync_error_count	sync_tale_error_count に変更

表 7 追加された information_schema スキーマ内のビュー

カタログ／ビュー名	説明
pg_edge_table_components	Property Graph オブジェクト情報
pg_element_table_key_columns	Property Graph オブジェクト・キー列情報
pg_element_table_labels	Property Graph オブジェクト・ラベル情報
pg_element_table_properties	Property Graph オブジェクト・属性情報
pg_element_tables	Property Graph オブジェクト・テーブル情報
pg_labels	Property Graph オブジェクト・ラベル情報
pg_property_graph_privileges	Property Graph オブジェクト権限情報
property_graphs	Property Graph 情報

表 8 内容が変更されたシステムカタログ／ビュー

カタログ／ビュー名	説明
pg_aggregate	aggtransspace 列は負数を許可する。array_agg 関数のように遷移状態が予測できない場合に使用される
pg_class	PROPERTY GRAPH オブジェクトを示す relkind='g'となるタプルが追加される
pg_subscription_rel	レプリケーション対象のシーケンスの情報が追加される
pg_stat_wal_receiver	status 列に接続中を示す「connecting」が出力される場合がある

□ pg_dsm_registry_allocations ビュー

pg_dsm_registry_allocations ビューは動的共有メモリー（DSM）の情報を提供します。このビューは SUPERUSER 属性を持つか、pg_read_all_stats ロールを持つユーザーのみ参照できます。 [\[167ed80\]](#)

表 9 pg_dsm_registry_allocations ビューの構造

列名	データ型	説明
name	text	エントリーの名前
type	text	エントリーのタイプを示します。以下の値が出力されます。 • segment • area • hash
size	bigint	エントリーのサイズ

例 14 pg_dsm_registry_allocations ビューの検索

```
postgres=# SELECT * FROM pg_dsm_registry_allocations ;
   name   | type   | size
-----+-----+-----
 autoprewarm | segment | 44
(1 row)
```

□ pg_stat_lock ビュー

pg_stat_lock ビューはロック時間の累積情報を取得できます。このビューは一般ユーザーも参照できます。[\[4019f72\]](#)

表 10 pg_stat_lock ビューの構造

列名	データ型	説明
locktype	text	ロック種別
waits	bigint	待機回数の累計
wait_time	bigint	待機時間の累計
fastpath_exceeded	bigint	高速パス経由でロックを取得できなかった回数
stats_reset	timestamp with time zone	統計リセット日時

例 15 pg_stat_lock ビューの検索

```
postgres=# SELECT locktype, waits, wait_time, fastpath_exceeded FROM
pg_stat_lock ;
```

locktype	waits	wait_time	fastpath_exceeded
relation	1	1000	0
extend	0	0	0
frozenid	0	0	0
page	0	0	0
tuple	0	0	0
transactionid	0	0	0
virtualxid	0	0	0
spectoken	0	0	0
object	0	0	0
userlock	0	0	0
advisory	0	0	0
applytransaction	0	0	0

(12 rows)

□ pg_stat_progress_data_checksums ビュー

pg_stat_progress_data_checksums ビューはチェックサム有効化の実行状況をリアルタイムに確認できます。[\[f19c0ec\]](#)

表 11 pg_stat_progress_data_checksums ビューの構造

列名	データ型	説明
pid	integer	ワーカーのプロセス ID
datid	oid	データベース OID
datname	name	データベース名
phase	text	実行フェーズ
databases_total	bigint	データベース総数
databases_done	bigint	処理済データベース数
relations_total	bigint	総テーブル数
relations_done	bigint	処理済テーブル数
blocks_total	bigint	総ブロック数
blocks_done	bigint	処理済ブロック数

例 16 pg_stat_progress_data_checksums ビューの検索

```
postgres=# SELECT * FROM pg_stat_progress_data_checksums
           WHERE datname='postgres' ;

-[ RECORD 1 ]-----
 pid          | 155779
 datid        | 5
 datname      | postgres
 phase        | enabling
 databases_total | 0
 databases_done | 0
 relations_total | 293
 relations_done | 0
 blocks_total  | 162163
 blocks_done   | 64146
```

□ pg_stat_progress_repack ビュー

pg_stat_progress_repack ビューは REPACK 文の実行状況をリアルタイムに確認できます。[\[ac58465\]](#)

表 12 pg_stat_progress_repack ビューの構造

列名	データ型	説明
pid	integer	サーバープロセス ID
datid	oid	データベース OID
datname	name	データベース名
relid	oid	REPACK 対象リレーション OID
command	text	実行 SQL (REPACK, VACUUM FULL 等)
phase	text	実行フェーズ文字列
repack_index_relid	oid	インデックス OID
heap_tuples_scanned	bigint	スキャンされたタプル数
heap_tuples_inserted	bigint	挿入されたタプル数
heap_tuples_updated	bigint	更新されたタプル数
heap_tuples_deleted	bigint	削除されたタプル数
heap_blks_total	bigint	テーブルのブロック数
heap_blks_scanned	bigint	スキャンされたブロック数
index_rebuild_count	bigint	インデックスのリビルド数

例 17 pg_stat_progress_repack ビューの検索

```
postgres=> SELECT * FROM pg_stat_progress_repack ;
```

```
-[ RECORD 1 ]-----+-----
```

```
pid           | 53203
datid         | 5
datname       | postgres
relid         | 16390
command       | REPACK
phase         | seq scanning heap
repack_index_relid | 0
heap_tuples_scanned | 4141595
heap_tuples_inserted | 4141595
heap_tuples_updated | 0
heap_tuples_deleted | 0
heap_blks_total | 54055
heap_blks_scanned | 22387
index_rebuild_count | 0
```

□ pg_stat_recovery ビュー

pg_stat_recovery ビューはスタンバイ・データベースのリカバリー状況を出力します。このビューは SUPERUSER 属性を指定されるか、pg_monitor ロールを持つユーザーが参照できます。[01d485b]

表 13 pg_stat_recovery ビューの構造

列名	データ型	説明
promote_triggered	boolean	プロモーションが発生したか
last_replayed_read_lsn	pg_lsn	リプレイされた開始 WAL 位置
last_replayed_end_lsn	pg_lsn	リプレイされた終了 WAL 位置
last_replayed_tli	integer	リプレイされた終了タイムライン
replay_end_lsn	pg_lsn	リプレイ中の WAL ロケーション
replay_end_tli	integer	リプレイ中の終了タイムライン
recovery_last_xact_time	timestamp with time zone	コミットされた最終トランザクション の日時
current_chunk_start_time	timestamp with time zone	リプレイが最新の受信 WAL チャンクに 追いついた日時
pause_state	text	リカバリー一時停止の状態

例 18 pg_stat_recovery ビューの検索

```
postgres=# SELECT * FROM pg_stat_recovery ;
-[ RECORD 1 ]-----+-----
promote_triggered      | f
last_replayed_read_lsn | 0/030005D8
last_replayed_end_lsn  | 0/03000610
last_replayed_tli      | 1
replay_end_lsn        | 0/03000610
replay_end_tli        | 1
recovery_last_xact_time | 2026-06-04 14:43:09.97718+09
current_chunk_start_time | 2026-06-04 14:45:43.83114+09
pause_state           | not paused
```

3.1.2. ロジカル・レプリケーションの拡張

ロジカル・レプリケーションには以下の新機能が実装されました。

□ シーケンスのレプリケーション

ロジカル・レプリケーションでシーケンスのレプリケーションがサポートされました。パブリケーション側では **ALL SEQUENCES** 句を指定してパブリケーションを作成します。パブリケーションに含まれるシーケンス名は `pg_publication_sequences` ビューに格納されます。[[5509055](#)]

例 19 パブリケーションの作成

```
postgres=# CREATE PUBLICATION pub1 FOR ALL SEQUENCES ;
CREATE PUBLICATION
postgres=# SELECT * FROM pg_publication_sequences ;
  pubname | schemaname | sequencename
-----+-----+-----
  pub1   | public    | seq01
(1 row)
```

サブスクライバ側ではレプリケーションを行うシーケンスをあらかじめ作成してサブスクリプションを作成します。対象となるシーケンスは `pg_subscription_rel` ビューに格納されます。

例 20 サブスクリプションの作成

```
postgres=# CREATE SUBSCRIPTION sub1 CONNECTION
          'host=dbsvr1 port=5432 dbname=postgres' PUBLICATION pub1 ;
NOTICE:  created replication slot "sub1" on publisher
CREATE SUBSCRIPTION
postgres=# SELECT * FROM pg_subscription_rel ;
 srsubid | srrelid | srsubstate | srsublsn
-----+-----+-----+-----
  16390 |  16389 | r          | 0/030659C0
(1 row)
```

シーケンス値を同期するにはサブスクライバ側で ALTER SUBSCRIPTION REFRESH SEQUENCES 文を実行します。

例 21 シーケンス値の同期

```
postgres=# SELECT setval('seq01', 1) ;
 setval
-----
      1
(1 row)

postgres=# ALTER SUBSCRIPTION sub1 REFRESH SEQUENCES ;
ALTER SUBSCRIPTION
postgres=# SELECT nextval('seq01') ;
 nextval
-----
      11
(1 row)
```

□ CREATE SUBSCRIPTION 文の SERVER 句

従来は PUBLICATION へ接続する情報を CREATE SUBSCRIPTION 文の CONNECTION 句に直接記述していました。PostgreSQL 19 では SERVER 句に既に作成済の SERVER オブジェクトを指定できるようになりました。[\[8185bb5\]](#)

例 22 SERVER 句の指定

```
postgres=# CREATE EXTENSION postgres_fdw ;
CREATE EXTENSION
postgres=# CREATE SERVER dbsvr1 FOREIGN DATA WRAPPER postgres_fdw OPTIONS
          (host 'dbsvr1', port '5432', dbname 'postgres') ;
CREATE SERVER
postgres=# CREATE USER MAPPING FOR PUBLIC SERVER dbsvr1 OPTIONS
          (user 'postgres', password '<<Password>>') ;
CREATE USER MAPPING
postgres=# CREATE SUBSCRIPTION sub1 SERVER dbsvr1 PUBLICATION pub1 ;
NOTICE:  created replication slot "sub1" on publisher
CREATE SUBSCRIPTION
```

□ 例外テーブルの指定

ALL TABLES 句を指定した PUBLICATION に対して例外とするテーブルを指定できます。CREATE PUBLICATION 文または ALTER PUBLICATION 文に EXCEPT 句を指定します。例外テーブルを示すために pg_publication_rel カタログに preexcept 列が追加されました。[[5984ea8](#), [fd36606](#), [493f8c6](#)]

例 23 例外テーブルの指定

```
postgres=# CREATE PUBLICATION pub1 FOR ALL TABLES EXCEPT (TABLE extable1) ;
CREATE PUBLICATION
postgres=# SELECT * FROM pg_publication_rel ;
   oid   | prpubid | prrelid | preexcept | prqual | prattrs
-----+-----+-----+-----+-----+-----
 16414   |   16413 |   16402 | t         |        |
(1 row)
```

□ サブスクリプションのオプション retain_dead_tuples

サブスクリプションにオプション retain_dead_tuples が追加されました。このオプションを有効にするとリモート・トランザクションが完了するまで削除済のタプルをサブスクライバ側で保持します。retain_dead_tuples オプションを true に設定する場合は、同時にパラメーター track_commit_timestamp を true に設定します (デフォルト false)。

[[228c370](#)]

例 24 retain_dead_tuples オプションの設定

```
postgres=# CREATE SUBSCRIPTION sub1 CONNECTION 'port=5432 dbname=postgres'
          PUBLICATION pub1 WITH (RETAIN_DEAD_TUPLES = TRUE) ;
NOTICE:  created replication slot "sub1" on publisher
CREATE SUBSCRIPTION
postgres=# SELECT subname, subretaindeadtuples FROM pg_subscription ;
-[ RECORD 1 ]-----+-----
subname          | sub1
subretaindeadtuples | t
```

□ サブスクリプションのオプション max_retention_duration

サブスクリプションにオプション `max_retention_duration` が追加されました。このオプションは `retain_dead_tuples` オプションが有効な環境で、削除不可能なトランザクション ID の最大保持期間を設定します。この閾値を超えると適用ワーカーは競合検出情報の保持を停止します。このオプションの設定は `pg_subscription` カタログの `submaxretention` 列で確認できます。[[a850be2](#)]

例 25 max_retention_duration オプションの設定

```
postgres=# CREATE SUBSCRIPTION sub1 CONNECTION 'port=5432 dbname=postgres'
          PUBLICATION pub1 WITH (MAX_RETENTION_DURATION = 10, RETAIN_DEAD_TUPLES =
          TRUE) ;
NOTICE:  created replication slot "sub1" on publisher
CREATE SUBSCRIPTION
postgres=# SELECT subname, submaxretention, subretentionactive FROM
          pg_subscription ;
subname | submaxretention | subretentionactive
-----+-----+-----
sub1    | 10              | t
(1 row)
```

□ デコード用メモリー量の検知

`pg_stat_replication_slots` ビューに `mem_exceeded_count` 列が追加されました。この列は論理デコード用のメモリーがパラメーター `logical_decoding_work_mem` を超えた回数を示します。[[d3b6183](#)]

例 26 `pg_stat_replication_slots` ビューの検索

```
postgres=# SELECT slot_name, mem_exceeded_count FROM
           pg_stat_replication_slots ;
-[ RECORD 1 ]-----+-----
slot_name      | sub1
mem_exceeded_count | 0
```

□ 衝突検知

サブスクリバ上の更新とローカルの削除が競合した場合に `update_deleted` 競合が検出できるようになりました。この機能を利用するにはサブスクリバに `retain_dead_tuples` オプションが有効になっている必要があります。[[fd5a1a0](#)]

□ WAL レベルの自動変更

パラメーター `wal_level` の設定がデフォルト値の `replica` の場合でも、論理レプリケーションスロットが作成されると WAL 出力のレベルが自動的に `logical` に変更されます。この状態を示すインターナル・パラメーター `effective_wal_level` が追加されました。すべての論理レプリケーションスロットが削除されると `effective_wal_level` パラメーターの値は元に戻ります。[[67c2097](#)]

例 27 WAL 出力レベルの自動アップデート

```
postgres=# SHOW wal_level ;
wal_level
-----
replica
(1 row)

postgres=# SHOW effective_wal_level ;
effective_wal_level
-----
replica
(1 row)

postgres=# SELECT * FROM
            pg_create_logical_replication_slot('slot1', 'test_decoding') ;
slot_name |    lsn
-----+-----
slot1     | 0/95033BD8
(1 row)

postgres=# SHOW effective_wal_level ;
effective_wal_level
-----
logical
(1 row)
```

3.1.3. パーティション・テーブルの拡張

パーティション・テーブルには以下の新機能が実装されました。

□ パーティションのマージ

RANGE/LIST パーティション・テーブルに対する ALTER TABLE 文で、複数のパーティションを単一のパーティションにマージできるようになりました。MERGE PARTITIONS 句に既存の複数パーティションを指定し、INTO 句にマージ先として新規作成されるパーティション名を指定します。[[f2e4cc4](#), [713e553](#)]

構文

```
ALTER TABLE table_name MERGE PARTITIONS (partition#1, partition#2, ...)
        INTO new_partition
```

下記の例ではパーティション `part1v1` と `part1v2` を `part1v12` パーティションにマージしています。

例 28 パーティションのマージ

```
postgres=> CREATE TABLE part1(c1 INT PRIMARY KEY, c2 VARCHAR(10))
            PARTITION BY RANGE(c1) ;
CREATE TABLE
postgres=> CREATE TABLE part1v1 PARTITION OF part1 FOR VALUES
            FROM (0) TO (1000000) ;
CREATE TABLE
postgres=> CREATE TABLE part1v2 PARTITION OF part1 FOR VALUES
            FROM (1000000) TO (2000000) ;
CREATE TABLE
postgres=> INSERT INTO part1 VALUES (generate_series(1, 1999999), 'data1') ;
INSERT 0 1999999
postgres=> ALTER TABLE part1 MERGE PARTITIONS (part1v1, part1v2) INTO part1v12 ;
ALTER TABLE
```

データの格納範囲が隣接しない RANGE パーティションはマージできません。

例 29 パーティションのマージ失敗

```
postgres=> CREATE TABLE part2(c1 INT PRIMARY KEY, c2 VARCHAR(10))
           PARTITION BY RANGE(c1) ;

CREATE TABLE
postgres=> CREATE TABLE part2v1 PARTITION OF part2 FOR VALUES
           FROM (0) TO (1000000) ;

CREATE TABLE
postgres=> CREATE TABLE part2v2 PARTITION OF part2 FOR VALUES
           FROM (1000000) TO (2000000) ;

CREATE TABLE
postgres=> CREATE TABLE part2v3 PARTITION OF part2
           FOR VALUES FROM (2000000) TO (3000000) ;

CREATE TABLE
postgres=> ALTER TABLE part2 MERGE PARTITIONS (part2v1, part2v3) INTO part2v13 ;
ERROR:  cannot merge partition "part2v3" together with partition "part2v1"
DETAIL:  The lower bound of partition "part2v3" is not equal to the upper bound
of partition "part2v1"
HINT:   ALTER TABLE ... MERGE PARTITIONS requires the partition bounds to be
adjacent.
```

□ パーティションの分割

RANGE/LIST パーティション・テーブルに対する ALTER TABLE 文でパーティションを複数のパーティションに分割できるようになりました。ALTER TABLE 文の SPLIT PARTITION 句に分割元のパーティションを指定し、INTO 句に分割先のパーティションとパーティション分割の値を指定します。[[4b3d173](#), [713e553](#)]

構文

```
ALTER TABLE table_name SPLIT PARTITION old_partition INTO
(PARTITION partition#1 VALUES ...,
 PARTITION partiotn#2 VALUES ...)
```


例 30 パーティションの分割

```
postgres=> CREATE TABLE part3(c1 INT PRIMARY KEY, c2 VARCHAR(10))
           PARTITION BY RANGE(c1) ;

CREATE TABLE
postgres=> CREATE TABLE part3v1 PARTITION OF part3 FOR VALUES
           FROM (0) TO (2000000) ;

CREATE TABLE
postgres=> INSERT INTO part3 VALUES (generate_series(1,1999999), 'data1') ;
INSERT 0 1999999

postgres=> ALTER TABLE part3 SPLIT PARTITION part3v1 INTO
           (PARTITION part3v2 FOR VALUES FROM (0) TO (1000000),
            PARTITION part3v3 FOR VALUES FROM (1000000) TO (2000000)) ;

ALTER TABLE
```

3.1.4. SQL Property Graph

PostgreSQL 19 にグラフ問合せ機能を提供する PROPERTY GRAPH オブジェクトが追加されました。この機能は Property Graph Queries (SQL/PGQ) - ISO/IEC 9075-16:2023 の実装です。PROPERTY GRAPH オブジェクトは CREATE PROPERTY GRAPH 文で作成します。グラフ問合せの元となるテーブルを作成し、PROPERTY GRAPH とマッピングを行います。VERTEX と NODE、EDGE と RELATIONSHIP はそれぞれ同じ意味です。本文書では詳細な使い方は説明していません。[[2f094e7](#)]

PROPERTY GRAPH の作成構文

```
CREATE PROPERTY GRAPH name
  [{VERTEX|NODE} TABLES (vertex_table_definition [, ...])]
  [{EDGE|RELATIONSHIP} TABLES (edge_table_definition [, ...])]
```

例 31 PROPERTY GRAPH の作成

```
postgres=> CREATE TABLE member (member_id INT PRIMARY KEY, member_name TEXT) ;
CREATE TABLE
postgres=> CREATE TABLE follow (member_id1 INT, member_id2 INTEGER,
PRIMARY KEY(member_id1, member_id2)) ;
CREATE TABLE
postgres=> CREATE PROPERTY GRAPH graph1
  VERTEX TABLES (member KEY (member_id) PROPERTIES (member_id, member_name))
  EDGE TABLES (follow
    SOURCE KEY (member_id1) REFERENCES member (member_id)
    DESTINATION KEY (member_id2) REFERENCES member (member_id)) ;
CREATE PROPERTY GRAPH
```

PROPERTY GRAPH 用に pg_prograph から始まる名前の複数のカタログが追加されました。PROPERTY GRAPH を検索するには GRAPH_TABLE 関数を使用します。

例 32 PROPERTY GRAPH の検索

```
postgres=> SELECT * FROM GRAPH_TABLE (graph1 MATCH
  (mem1 IS member)-[follow1 IS follow]->(mem2 IS member)
  -[follow2 IS follow]->(mem3 IS member)
  WHERE mem1.member_id = mem3.member_id COLUMNS
    (mem1.member_id, mem2.member_id)) ;
member_id | member_id
-----+-----
100 | 200
200 | 100
(2 rows)
```

3.1.5. SSL/SNI

PostgreSQL サーバー側で SSL Server Name Indication (SNI) をサポートするようになりました。この機能はパラメーター ssl_sni を on に設定することで有効化されます (デフォルト値 off)。SNI 機能の設定を行うためにデータベース・クラスタの pg_hosts.conf ファイルを使用します。pg_hosts.conf ファイルには以下の項目を 1 行毎に記述します。[4f43302]

表 14 pg_hosts.conf ファイルの内容

項目	説明
HOSTNAME	ホスト名（必須）
SSL CERTIFICATE	SSL 証明書ファイル（必須）
SSL KEY	SSL 秘密鍵ファイル（必須）
SSL CA	SSL 認証ファイル（オプション）
PASSPHRASE COMMAND	パスフレーズ（オプション）
PASSPHRASE COMMAND RELOAD	パスフレーズを再読み込みするか（オプション）

3.1.6. チェックサムの動的変更

ブロックのチェックサム機能をインスタンス起動状態で有効化／無効化できるようになりました。有効化する場合には `pg_enable_data_checksums` 関数を、無効化する場合には `pg_disable_data_checksums` 関数を実行します。チェックサムの有効化はバックグラウンドワーカー・プロセスによって実行されます。`pg_enable_data_checksums` 関数には `cost_delay` パラメーター（デフォルト値 0）と、`cost_limit` パラメーター（デフォルト値 100）を指定でき、VACUUM のコスト遅延と同様の動作でチェックサムの有効化を行います。チェックサム有効化はバックグラウンドで行われ、進行状況は `pg_stat_progress_data_checksums` ビューで確認できます。[\[f19c0ec\]](#)

例 33 チェックサムの動的変更

```
postgres=# SHOW data_checksums ;
-[ RECORD 1 ]--+-
data_checksums | on

postgres=# SELECT pg_disable_data_checksums() ;
-[ RECORD 1 ]-----+
pg_disable_data_checksums |

postgres=# SHOW data_checksums ;
-[ RECORD 1 ]--+-
data_checksums | off

postgres=# SELECT pg_enable_data_checksums(1, 110) ;
-[ RECORD 1 ]-----+
pg_enable_data_checksums |

postgres=# SHOW data_checksums ;
-[ RECORD 1 ]--+-
data_checksums | inprogress-on
```

3.1.7. ロール

ロール `pg_read_all_data`、`pg_write_all_data` はラージオブジェクトの読み書きに対応しました。旧バージョンでは「`ERROR: permission denied for large object {OID}`」エラーが発生していました。[\[d981976\]](#)

例 34 ラージオブジェクトの参照権限

```
postgres=# SELECT lo_create(-1) ;
  lo_create
-----
 4294967295
(1 row)

postgres=# SELECT lo_put(4294967295, 1, E'¥xaa') ;
  lo_put
-----

(1 row)

postgres=# GRANT pg_read_all_data TO user1 ;
GRANT ROLE
postgres=# \connect postgres user1
You are now connected to database "postgres" as user "user1".
postgres=> SELECT lo_get(4294967295) ;
  lo_get
-----
 ¥x00aa
(1 row)
```

3.1.8. Libpq

以下の API が追加・拡張されました。

□ 接続文字列 servicefile

接続文字列にサービス用ファイルを指定する `servicefile` が追加されました。従来通り環境変数 `PGSERVICEFILE` も利用できます。[092f3c6]

例 35 接続文字列 servicefile

```
$ psql "servicefile=/tmp/serviceA.conf service=service1"
psql (19beta1)
Type "help" for help.

postgres=#
```

□ 接続文字列 oauth_ca_file

接続文字列に OAuth 証明書を指定する `oauth_ca_file` が追加されました。環境変数 `PGOAUTHCAFILE` はデバッグモード以外でも使えるようになりました。[[9933681](#)]

□ バージョン情報の取得

`PG_VERSION` ファイルからバージョン情報を取得する `get_pg_version` 関数が追加されました。この関数は `pg_resetwal`, `pg_combinebackup`, `pg_createsubscriber`, `pg_upgrade` 等で使用されています。[[cd0be13](#)]

□ その他の API

以下の API が追加されました。[[fe07100](#), [7508169](#), [e533524](#), [b8d7685](#)]

表 15 追加された API 名

API 名	説明
<code>pg_database_locale</code>	データベースのデフォルト・ロケールを取得
<code>GetNamedDSA</code>	DSA の割り当てを行う
<code>GetNamedDSHash</code>	DSA ハッシュの割り当てを行う
<code>ParallelSlotSetIdle</code>	<code>ParallelSlot</code> をアイドル状態にマークする
<code>PQgetThreadLock</code>	カレントスレッドのロック情報を取得する

3.1.9. I/O 処理

PostgreSQL 19 の I/O 処理の改善点です。

□ 非同期 I/O ワーカーの自動調整

I/O ワーカー数が負荷に応じて自動的に増減するようになりました。稼働する I/O ワーカー数は以下のパラメーターで決定されます。I/O キューが増大するとワーカー数が増加し、一定時間アイドル状態になるとワーカーが停止します。[[d1c01b7](#)]

表 16 I/O ワーカーに関するパラメーター

パラメーター名	説明	デフォルト値
io_min_workers	最小 I/O ワーカー数	2
io_max_workers	最大 I/O ワーカー数	8
io_worker_idle_timeout	ワーカーが終了するアイドル時間	60s
io_worker_launch_interval	ワーカーが起動される間隔	100ms

□ ストリーム I/O の拡大

PostgreSQL 19 ではストリーム I/O の範囲が拡大しました。

- BRIN インデックスの VACUUM [[a6eac22](#)]
- ハッシュインデックスのデータ削除読み込み [[bfa3c4f](#)]
- Contrib モジュールの bloom によるインデックス読み込み [[4c910f3](#), [d841ca2](#)]
- Contrib モジュールの pgstattuple によるインデックス読み込み [[213f007](#)]

□ 非同期処理の改善

I/O メソッドに io_uring を使用する場合、4 ブロック以下の読み込みでは同期 I/O を使うように変更されました。 [[a9ee668](#), [8ca147d](#), [f63ca33](#)]

3.1.10. 自動 VACUUM

自動 VACUUM 処理には以下の機能が追加されました。

□ パラレル・ワーカーの使用

自動 VACUUM 実行時にパラレル・ワーカーを使用できるようになりました。この機能は自動 VACUUM で VACUUM (PARALLEL)文と同じ処理を行います。並列度はパラメーターautovacuum_max_parallel_workers で決まります。このパラメーターのデフォルト値は 0 です。自動 VACUUM のログからパラレル・ワーカーが起動していることが確認できます。 [[1ff3180](#), [2a3d2f9](#)]

例 36 自動 VACUUM のログ

```
LOG:  automatic vacuum of table "postgres.public.data1": index scans: 1
...
index scan needed: 5406 pages from table (100.00% of total) had 500000 dead ...
parallel workers: index vacuum: 1 planned, 1 launched in total
...
```

□ スコアリングによる優先順位

PostgreSQL 19 に自動 VACUUM の実行順序を決めるスコアリングシステムが導入されました。VACUUM や ANALYZE の実行を決める閾値から最も大きく逸脱しているテーブルを優先して処理します。スコアは新規追加された `pg_stat_autovacuum_scores` ビューで決定されます。例えば UPDATE 文により 100 タプルが更新され、VACUUM の閾値が 80 の場合、スコアは 1.25 になります。実際のスコアはウェイトを示す下記のパラメーターが乗算されて決定されます。[[87f61f0](#)]

表 17 スコアに関連するパラメーター

パラメーター名	デフォルト値	説明
<code>autovacuum_analyze_score_weight</code>	1.0	ANALYZE のウェイト
<code>autovacuum_freeze_score_weight</code>	1.0	FREEZE のウェイト
<code>autovacuum_multixact_freeze_score_weight</code>	1.0	マルチトランザクション FREEZE のウェイト
<code>autovacuum_vacuum_insert_score_weight</code>	1.0	VACUUM INSRT のウェイト
<code>autovacuum_vacuum_score_weight</code>	1.0	VACUUM のウェイト

`pg_stat_autovacuum_scores` ビューの構造は以下の通りです。

表 18 `pg_stat_autovacuum_scores` ビューの構造

列名	データ型	説明
<code>relid</code>	<code>oid</code>	テーブル OID
<code>schemaname</code>	<code>name</code>	スキーマ名
<code>relname</code>	<code>name</code>	テーブル名
<code>score</code>	<code>double precision</code>	スコア
<code>xid_score</code>	<code>double precision</code>	トランザクション ID のスコア
<code>vacuum_score</code>	<code>double precision</code>	VACUUM のスコア
<code>vacuum_insert_score</code>	<code>double precision</code>	INSERT の VACUUM スコア
<code>analyze_score</code>	<code>double precision</code>	ANALYZE のスコア
<code>do_vacuum</code>	<code>boolean</code>	VACUUM を実行するか
<code>do_analyze</code>	<code>boolean</code>	ANALYZE を実行するか
<code>for_wraparound</code>	<code>Boolean</code>	トランザクション周回のため VACUUM を行う

例 37 pg_stat_autovacuum_scores ビューの検索

```
postgres=> SELECT * FROM pg_stat_autovacuum_scores WHERE relname='data1' ;
-[ RECORD 1 ]-----+-----
relid          | 16385
schemaname     | public
relname        | data1
score          | 9.990009990000999
xid_score      | 2e-08
mxid_score     | 0
vacuum_score   | 4.997501249375312
vacuum_insert_score | 0
analyze_score  | 9.990009990000999
do_vacuum      | t
do_analyze     | t
for_wraparound | f
```

3.1.11. フック

プランナーに対する以下のフックが追加されました。

□ planner_setup_hook、planner_shutdown_hook

PlannerGlobal オブジェクトが初期化される時点に実行される planner_setup_hook フック、オブジェクトが破棄される直前に実行される planner_shutdown_hook フックが追加されました。[[94f3ad3](#), [0f4c8d3](#)]

□ joinrel_setup_hook、join_path_setup_hook

テーブルの結合情報(joinrel)の作成時に実行されるフック joinrel_setup_hook と、joinrel にパスを追加する時に実行されるフック join_path_setup_hook が追加されました。[[4020b37](#)]

3.1.12. 高速化

プロセッサ命令やソート処理の変更による高速化が実装されました。

□ ソート処理

一部のソート処理が Quick ソートから Radix ソート（基数ソート）に変更されました。アイテム数が 40 個未満場合は引き続き Quick ソートが行われます。[[ef3c3cf](#)]

□ hex_encode / hex_decode 内部関数

hex_encode 関数、hex_decode 関数は x86-64 アーキテクチャおよび Aarch64 アーキテクチャの SIMD 命令を使用できるようになりました。入力値が大きいほど効果が期待できます。[[ec8719c](#)]

□ ページ・チェックサム

利用できる場合はページ・チェックサムの計算に AVX2 インストラクションを使用します。[[5e13b0f](#)]

□ CRC32C

Arm v8 プロセッサで Crypto Extension が利用できる場合、CRC32C 計算に使用されます。[[fbc57f2](#)]

□ COPY FROM 文

SIMD 命令を使って特殊文字の有無をチェックすることで高速化を行うようになりました。[[e0a3a3f](#)]

3.1.13. ログ出力

自動 ANALYZE のログ出力にも Full Page イメージのバイト数が出力されるようになりました。[[ad25744](#)]

例 38 自動 ANALYZE のログ

```
LOG:  automatic analyze of table "postgres.public.data1"
      avg read rate: 1.486 MB/s, avg write rate: 0.255 MB/s
      buffer usage: 10994 hits, 35 reads, 6 dirtied
      WAL usage: 14 records, 3 full page images, 27293 bytes, 24576 full page
      image bytes, 0 buffers full
      system usage: CPU: user: 0.03 s, system: 0.01 s, elapsed: 0.18 s
```

3.1.14. OAuth 認証

OAuth 認証機能には以下の新機能が追加されました。

□ pg_hba.conf ファイル設定

pg_hba.conf のオプションに OAuth バリデーターに関する情報を記述できるようになりました。以下はマニュアルに記載された pg_hba.conf の例です。[b977bd3]

例 39 pg_hba.conf ファイルの OAuth 設定

```
hostssl postgres admin 0.0.0.0/0 oauth issuer=https://admin.example.com ¥
scope="pg-admin openid email" ¥
map=oauth-email ¥
validator=my_validator ¥
validator.foo="admin access" ¥
validator.bar=https://magic.example.com
```

□ バリデーターのエラー出力

バリデーターで発生したエラーのログ出力方法について以下のパラメーターが追加されました。[d438a36]

表 19 追加されたパラメーター

パラメーター名	説明	デフォルト値
oauth_validator.error_detail	DETAIL に含まれるエラー情報	"
oauth_validator.internal_error	内部エラーを出力するか	false

3.1.15. 待機イベント

以下の待機イベントが追加されました。[6c5c393, e05a24c, 7a39f43]

表 20 追加された待機イベント

待機イベント名	説明	備考
WaitEventBuffer	データバッファ待ち	WaitEventBufferPin から変更
COPY_FROM_READ	COPY FROM 読み込み待ち	
COPY_TO_WRITE	COPY TO 書き込み待ち	
WAIT_FOR_WAL_WRITE	スタンバイへの書き込み待ち	

3.2. SQL 文の拡張

本セクションでは SQL 文に関する新機能を説明しています。

3.2.1. ALTER DEFAULT PRIVILEGE 文

ラージオブジェクトに対してデフォルト権限を設定できるようになりました。
pg_default_acl カタログの defaclobjtype 列にはラージオブジェクトを示す L が出力されます。[0d6c477]

構文

```
ALTER DEFAULT PRIVILEGES GRANT {SELECT|UPDATE} ON LARGE OBJECTS TO role_name
```

例 40 LARGE OBJECT のデフォルト権限

```
postgres=> ALTER DEFAULT PRIVILEGES GRANT SELECT ON LARGE OBJECTS TO PUBLIC ;
ALTER DEFAULT PRIVILEGES
postgres=> SELECT defaclobjtype, defaclacl FROM pg_default_acl ;
 defaclobjtype |      defaclacl
-----+-----
L              | {=r/demo, demo=rw/demo}
(1 row)
```

3.2.2. ALTER DOMAIN 文

ALTER DOMAIN VALIDATE CONSTRAINT 文の実行に必要なロックレベルが SHARE から SHARE UPDATE EXCLUSIVE に変更されました。[16a0039]

3.2.3. ALTER FOREIGN DATA WRAPPER 文

新規の接続を行う関数を指定する CONNECTION 句が追加されました。[8185bb5]

構文

```
ALTER FOREIGN DATA WAPPER fdw_name CONNECTION function_name
ALTER FOREIGN DATA WAPPER fdw_name NO CONNECTION
```

3.2.4. ALTER PUBLICATION 文

ALTER PUBLICATION 文に SET ALL TABLES 句と SET ALL SEQUENCES 句が指定できるようになりました。この指定ができるのは ALL TABLES/SEQUENCES 句を指定したパブリケーションか、空のパブリケーションの場合だけです。[[5984ea8](#), [fd36606](#), [493f8c6](#), [5509055](#)]

構文

```
ALTER PUBLICATION publication_name SET ALL TABLES [EXCEPT (TABLE expression)]  
ALTER PUBLICATION publication_name SET ALL SEQUENCES
```

例 41 ALTER PUBLICATION 文の実行

```
postgres=# CREATE PUBLICATION pub1 FOR ALL TABLES ;  
CREATE PUBLICATION  
postgres=# ALTER PUBLICATION pub1 SET ALL TABLES EXCEPT (TABLE data2) ;  
ALTER PUBLICATION  
postgres=# CREATE PUBLICATION pub2 ;  
CREATE PUBLICATION  
postgres=# ALTER PUBLICATION pub2 SET ALL SEQUENCES ;  
ALTER PUBLICATION
```

3.2.5. ALTER SUBSCRIPTION 文

サブスクリプションに含まれるシーケンス情報を更新する ALTER SUBSCRIPTION REFRESH SEQUENCES 文が追加されました。[[f0b3573](#)]

構文

```
ALTER SUBSCRIPTION subscription_name REFRESH SEQUENCES
```

例 42 REFRESH SEQUENCES 句の指定

```
postgres=# ALTER SUBSCRIPTION sub1 REFRESH SEQUENCES ;  
ALTER SUBSCRIPTION
```

3.2.6. ALTER TABLE 文

ALTER TABLE 文には以下の拡張が実装されました。

□ 仮想生成列の計算式変更

CHECK 制約が指定された仮想生成列に対して SET EXPRESSION 句で計算式を変更できるようにになりました。[\[f80bedd\]](#)

構文

```
ALTER TABLE table_name ALTER COLUMN column_name SET EXPRESSION AS expression
```

例 43 CHECK 制約付き仮想生成列の変更

```
postgres=> CREATE TABLE data1(col1 INT, col2 INT GENERATED ALWAYS AS
              (col1 * 100) CHECK (col2 < 10000)) ;
CREATE TABLE
postgres=> ALTER TABLE data1 ALTER COLUMN col2 SET EXPRESSION AS
              (col1 * 200) ;
ALTER TABLE
```

□ CHECK 制約の ENFOCED 設定

CHECK 制約に対して ENFORCED 句の設定を変更できるようになりました。[\[342051d\]](#)

構文

```
ALTER TABLE table_name ALTER CONSTRAINT constraint_name [NOT] ENFORCED
```

例 44 CHECK 制約の ENFORCED 句設定

```
postgres=> CREATE TABLE data1(col1 INT PRIMARY KEY, col2 INT) ;
CREATE TABLE
postgres=> ALTER TABLE data1 ADD CONSTRAINT data1_col2_check1
        CHECK(col2 < 1000) ;
ALTER TABLE
postgres=> INSERT INTO data1 SELECT g1, g1 FROM generate_series(1, 999) g1 ;
INSERT 0 999
postgres=> ALTER TABLE data1 ALTER CONSTRAINT data1_col2_check1 NOT ENFORCED ;
ALTER TABLE
postgres=> INSERT INTO data1 VALUES (1000, 1000) ;
INSERT 0 1
postgres=> ALTER TABLE data1 ALTER CONSTRAINT data1_col2_check1 ENFORCED ;
ERROR:  check constraint "data1_col2_check1" of relation "data1" is violated by
some row
postgres=> DELETE FROM data1 WHERE col2=1000 ;
DELETE 1
postgres=> ALTER TABLE data1 ALTER CONSTRAINT data1_col2_check1 ENFORCED ;
ALTER TABLE
```

□ オプション autovacuum_parallel_workers

テーブルのオプション autovacuum_parallel_workers が追加されました。このオプションは CREATE TABLE 文でも指定できます。VACUUM パラレルワーカースレッド数を指定します。オプションのデフォルト値は -1 で、パラメーター autovacuum_max_parallel_workers に従います。このオプションを 0 に設定すると、パラレル VACUUM が無効になります。[[1ff3180](#)]

例 45 自動 VACUUM パラレル・ワーカー数の設定

```
postgres=> ALTER TABLE data1 SET (autovacuum_parallel_workers = 4) ;
ALTER TABLE
```

□ ALTER TABLE 文の高速化

ALTER TABLE ADD COLUMN 文で、追加される列が CHECK 制約または NOT NULL 制約付きドメインである場合、書き換えの必要性をチェックできるようになりました。書き換え不要な場合、テーブルの書き換えを行いません。[a0b6ef2]

3.2.7. CHECKPOINT 文

CHECKPOINT 文には以下のオプションが追加されました。

□ MODE オプション

MODE オプションはチェックポイントの実行方法を選択します。MODE FAST オプションはデフォルトの動作であり、できるだけ高速にチェックポイントを実行します。MODE SPREAD オプションを指定すると、チェックポイントに関連するパラメーターを元に負荷を下げて実行します。[2f698d7]

構文

```
CHECKPOINT [(MODE {FAST|SPREAD})]
```

例 46 MODE オプション

```
postgres=# CHECKPOINT (MODE FAST) ;
CHECKPOINT
postgres=# CHECKPOINT (MODE SPREAD) ;
CHECKPOINT
```

このオプションの指定により、出力されるログが以下のように変化します。

表 21 出力されるログ

オプション	ログメッセージ
FAST	LOG: checkpoint starting: fast force wait
SPREAD	LOG: checkpoint starting: force wait

□ FLUSH_UNLOGGED オプション

FLUSH_UNLOGGED オプションを指定すると、チェックポイントでログに記録されていないダーティバッファもフラッシュします。[8d33fba, a4f1265]

構文

```
CHECKPOINT [(FLUSH_UNLOGGED [boolean])]
```

例 47 FLUSH_UNLOGGED オプション

```
postgres=# CHECKPOINT (FLUSH_UNLOGGED) ;  
CHECKPOINT
```

オプションの指定により、出力されるログが以下のように変化します。

表 22 出力されるログ

オプション	ログメッセージ
指定無し	LOG: checkpoint starting: fast force wait
FLUSH_UNLOGGED	LOG: checkpoint starting: fast force wait flush-unlogged

3.2.8. COPY 文

COPY 文には以下の新機能が実装されました。

□ ON_ERROR オプション設定値

COPY FROM 文の `on_error` オプションに `set_null` を指定できるようになりました。このオプションはデータ型変換エラーが発生した列値に `NULL` を指定します。[[2a525cc](#)]

例 48 ON_ERROR オプション

```
postgres=# \d data1
               Table "public.data1"
  Column | Type   | Collation | Nullable | Default
-----+-----+-----+-----+-----
 col1   | integer |           |          |
 col2   | integer |           |          |

postgres=# COPY data1 FROM stdin WITH (format 'csv', on_error 'set_null') ;
Enter data to be copied followed by a newline.
End with a backslash and a period on a line by itself, or an EOF signal.
>> 100,100
>> 200,bad_data
>> 300,300
>> \.
NOTICE:  in 1 row, columns were set to null due to data type incompatibility
COPY 3
postgres=# SELECT * FROM data1 ;
 col1 | col2
-----+-----
  100 |  100
  200 | null
  300 |  300
(3 rows)
```

□ パーティション・テーブル対応

パーティション・テーブルに対する COPY TO 文が実行できるようになりました。旧バージョンではパーティション・テーブル名ではなく COPY (SELECT ...) TO 文を実行する必要がありました。[[4bea91f](#)]

例 49 パーティション・テーブルの指定

```
postgres=# \d part1
                Partitioned table "public.part1"
  Column |          Type          | Collation | Nullable | Default
-----+-----+-----+-----+-----
  col    | integer                |           | not null |
  col2   | character varying(10) |           |          |
Partition key: RANGE (c1)
Indexes:
    "part1_pkey" PRIMARY KEY, btree (col1)
Number of partitions: 2 (Use \d+ to list them.)

postgres=# COPY part1 TO '/tmp/part1.csv' CSV ;
COPY 2000000
```

□ ヘッダー行数

データ内のヘッダー行数を指定できるようになりました。[[bc2f348](#)]

例 50 ヘッダー行数の指定

```
postgres=# COPY data1 FROM stdin WITH (FORMAT CSV, HEADER 2) ;
Enter data to be copied followed by a newline.
End with a backslash and a period on a line by itself, or an EOF signal.
>> HEADER1
>> HEADER2
>> 100,data1
>> 200,data2
>> \.
COPY 2
```

□ JSON フォーマット出力

COPY TO 文の FORMAT オプションに json を指定できるようになりました。追加オプションとして、force_array を指定できます。[[7dadd38](#), [4c0390a](#)]

例 51 JSON フォーマット出力

```
postgres=# COPY data1 TO stdout WITH (FORMAT json) ;
{"col1":1,"col2":"data1"}
{"col1":2,"col2":"data2"}
{"col1":3,"col2":"data3"}
{"col1":4,"col2":"data4"}
{"col1":5,"col2":"data5"}
postgres=# COPY data1 TO stdout WITH (FORMAT json, FORCE_ARRAY) ;
[
  {"col1":1,"col2":"data1"}
, {"col1":2,"col2":"data2"}
, {"col1":3,"col2":"data3"}
, {"col1":4,"col2":"data4"}
, {"col1":5,"col2":"data5"}
]
```

3.2.9. CREATE PUBLICATION 文

全シーケンスを対象とする FOR ALL SEQUENCES 句が指定できるようになりました。この句を指定した CREATE PUBLICATION 文の実行には SUPERUSER 属性を持つユーザーが必要です。pg_publication カタログにはこの句を指定したかを示す puballsequences 列が追加されています。[\[96b3784\]](#)

構文

```
CREATE PUBLICATION publication_name FOR ALL SEQUENCES
```

例 52 FOR ALL SEQUENCES 句の指定

```
postgres=# CREATE PUBLICATION pub1 FOR ALL SEQUENCES ;
```

```
CREATE PUBLICATION
```

```
postgres=# \drp
```

```
                                List of publications
Name | Owner  | All tables | All sequences | Inserts | Updates | Deletes...
-----+-----+-----+-----+-----+-----+-----
pub1 | postgres | f          | t              | t        | t        | t        ...
(1 row)
```

ALL SEQUENCES 句を指定したパブリケーションにはテーブルを追加することはできません。

例 53 パブリケーションにテーブル追加

```
postgres=# ALTER PUBLICATION pub1 ADD TABLE data1 ;
```

```
ERROR: publication "pub1" is defined as FOR ALL SEQUENCES
```

```
DETAIL: Tables or sequences cannot be added to or dropped from FOR ALL SEQUENCES
publications.
```

3.2.10. CREATE SCHEMA 文

CREATE SCHEMA 文にテーブル、ビュー、インデックス、シーケンス、トリガー、GRANT 以外のオブジェクトを指定できるようになりました。[[a9c350d](#), [404db8f](#)]

例 54 CREATE SCHEMA 文とオブジェクトの作成

```
postgres=# CREATE SCHEMA schema1
```

```
        CREATE TABLE data1(col1 INT PRIMARY KEY, col2 VARCHAR(10))
```

```
        CREATE COLLATION col1 (LOCALE="C")
```

```
        CREATE DOMAIN d1 AS INT ;
```

```
CREATE SCHEMA
```

指定できる CREATE 句は以下の通りです。

表 23 同時作成できるオブジェクトの種類

オブジェクト	PostgreSQL 18	PostgreSQL 19
AGGREGATE	-	○
COLLATION	-	○
DOMAIN	-	○
FUNCTION	-	○
INDEX	○	○
OPERATOR	-	○
PROCEDURE	-	○
SEQUENCE	○	○
TABLE	○	○
TEXT SEARCH CONFIGURATION	-	○
TEXT SEARCH DICTIONARY	-	○
TEXT SEARCH PARSER	-	○
TEXT SEARCH TEMPLATE	-	○
TRIGGER	○	○
TYPE	-	○
VIEW	○	○
GRANT	○	○

3.2.11. CREATE STATISTICS 文

仮想生成列を指定して、拡張統計を作成できるようになりました。[f7f4052]

例 55 仮想生成列に対する拡張統計の作成

```
postgres=> CREATE TABLE data1(col1 INT, col2 INT GENERATED ALWAYS AS
              (col1 + 100) VIRTUAL) ;

CREATE TABLE
postgres=> CREATE STATISTICS data1_stat1 ON (col2) FROM data1 ;
CREATE STATISTICS
```

3.2.12. DELETE / UPDATE 文

UPDATE 文と DELETE 文に FOR PORTION OF 句が指定できるようになりました。FOR PORTION OF 句に範囲を指定すると、範囲に合致する新規タプルが作成されて SET 句に指定された更新が行われます。元のタプルは FOR PORTION OF 句の範囲以外のタプルに分割されます。[\[8e72d91\]](#)

構文

```
UPDATE table_name FOR PORTION OF column FROM start TO end SET ... WHERE ...  
DELETE FROM table_name FOR PORTION OF column FROM start TO end WHERE ...
```

動作がわかりにくいので例を使って解説します。当初 `range1` テーブルには 2 タプルが格納されています。DATERANGE 型の `col1` 列に「2026/04/01～2026/05/01」を示すタプルと、「2026/05/01～2026/06/01」を示すタプルです。

例 56 更新前のタプル

```
postgres=> SELECT * FROM range1 ORDER BY col1 ;  
          col1          | col2  
-----+-----  
 [2026-04-01, 2026-05-01) | 100  
 [2026-05-01, 2026-06-01) | 200  
(2 rows)
```

FOR PORTION OF 句に「2026/04/10～2026/04/20」を指定すると、更新対象のタプルが選択されます (`col2=100` のタプル)。このタプルは以下の 3 タプルに分割されます。

- 2026/04/01～2026/04/10 (元のタプル範囲の前半)
- 2026/04/10～2026/04/20 (更新対象となるタプル、SET 句で更新)
- 2026/04/20～2026/05/01 (元のタプル範囲の後半)

例 57 更新後のタプル

```
postgres=> UPDATE range1 FOR PORTION OF col1 FROM '2026-04-10' TO '2026-04-20'
           SET col2=300 ;

UPDATE 1
postgres=> SELECT * FROM range1 ORDER BY col1 ;
```

col1	col2
[2026-04-01, 2026-04-10)	100
[2026-04-10, 2026-04-20)	300
[2026-04-20, 2026-05-01)	100
[2026-05-01, 2026-06-01)	200

(4 rows)

3.2.13. EXPLAIN 文

EXPLAIN 文には以下の機能が追加されました。

□ I/O 統計の出力

EXPLAIN 文に I/O 統計を出力する IO オプションが追加されました。IO オプションは ANALYZE オプションと同時に指定します。[[681daed](#), [3b1117d](#), [e157fe6](#)]

例 58 EXPLAIN (IO)文の実行

```
postgres=> EXPLAIN (ANALYZE, IO, COSTS OFF) SELECT COUNT(*) FROM data1 ;
           QUERY PLAN
```

```
Aggregate (actual time=425.023..425.024 rows=1.00 loops=1)
  Buffers: shared hit=1099 read=52956
  -> Seq Scan on data1 (actual time=0.416..235.808 rows=10000000.00 loops=1)
        Prefetch: avg=70.84 max=91 capacity=94
        I/O: count=3314 waits=7 size=15.98 in-progress=4.54
        Buffers: shared hit=1099 read=52956
Planning Time: 0.059 ms
Execution Time: 425.056 ms
(8 rows)
```


□ Memoize 見積もり情報の出力

実行計画に Memoize ノードが含まれる場合に見積もり情報 (Estimates:) が出力されるようになりました。[4bc62b8]

例 59 Memoize 見積もり情報の出力

```
postgres=> EXPLAIN SELECT * FROM data1 AS d1 INNER JOIN data1 AS d2
           ON d1.col1 = d2.col2::numeric AND d1.col2::numeric = d2.col1 ;
           QUERY PLAN
-----
Nested Loop  (cost=0.15..10.49 rows=1 width=36)
-> Seq Scan on data1 d1  (cost=0.00..1.40 rows=40 width=18)
-> Memoize  (cost=0.15..0.41 rows=1 width=18)
    Cache Key: d1.col1, (d1.col2)::numeric
    Cache Mode: logical
    Estimates: capacity=20 distinct keys=20 lookups=40 hit percent=50.00%
-> Index Only Scan using idx1_data1 on data1 d2  (cost=0.14..0.40...
    Index Cond: (col1 = (d1.col2)::numeric)
    Filter: (d1.col1 = (col2)::numeric)

(9 rows)
```

□ WAL FPI バイト数の出力

出力される WAL 情報に FPI バイトが含まれるようになりました。[5ab0b6a] [5ab0b6a]

例 60 FPI バイト情報の出力

```
postgres=> EXPLAIN (ANALYZE, WAL) UPDATE data1 SET col2='update'
           WHERE col1 < 100 ;

               QUERY PLAN

-----
Update on data1 (cost=0.00..1.50 rows=0 width=0) (actual time=0.225..0.225...
  Buffers: shared hit=121 read=1 dirtied=3
  WAL: records=80 fpi=2 bytes=9790 fpi bytes=3656
  -> Seq Scan on data1 (cost=0.00..1.50 rows=40 width=38) (actual time=0.000..0.000...
    Filter: (col1 < '100'::numeric)
    Buffers: shared hit=1
Planning Time: 0.281 ms
Execution Time: 0.363 ms
(8 rows)
```

□ サブクエリー名称

出力される実行計画のサブクエリー・ブロックに一意な名前が出力されるようになりました。従来は番号のみの出力でした。[[8c49a48](#)]

例 61 PostgreSQL 19 の出力

```
postgres=> EXPLAIN (COSTS OFF) SELECT COUNT(*) FROM data1 WHERE col1 =
           (SELECT MAX(col1) FROM data1) ;

               QUERY PLAN

-----
Aggregate
  InitPlan expr_1
    -> Result
      Replaces: MinMaxAggregate
      InitPlan minmax_1
        -> Limit
          -> Index Only Scan Backward using data1_pkey on data1
data1_1
    -> Index Only Scan using data1_pkey on data1
      Index Cond: (col1 = (InitPlan expr_1).col1)
(9 rows)
```

例 62 PostgreSQL 18 の出力

```
postgres=> EXPLAIN (COSTS OFF) SELECT COUNT(*) FROM data1 WHERE col1 =  
           (SELECT MAX(col1) FROM data1) ;  
           QUERY PLAN  
  
-----  
Aggregate  
  InitPlan 2  
    -> Result  
      InitPlan 1  
        -> Limit  
          -> Index Only Scan Backward using data1_pkey on data1  
data1_1  
  -> Index Only Scan using data1_pkey on data1  
      Index Cond: (col1 = (InitPlan 2).col1)  
(8 rows)
```

3.2.14. GRANT 文

GRANT 文と REVOKE 文に権限付与者を示す GRANTED BY 句にカレント・ユーザー以外を指定できるようになりました。従来のバージョンでは「ERROR: grantor must be current user」エラーが発生していました。[\[dd1398f\]](#)

例 63 GRANTED BY 句の指定

```
postgres=> CREATE TABLE data1(col1 INT PRIMARY KEY, col2 VARCHAR(10)) ;
CREATE TABLE
postgres=> GRANT SELECT ON data1 TO demo2 WITH GRANT OPTION ;
GRANT
postgres=> \connect postgres postgres
You are now connected to database "postgres" as user "postgres".
postgres=# GRANT SELECT ON data1 TO demo3 GRANTED BY demo2 ;
GRANT
postgres=# \dp+
```

Access privileges					
Schema	Name	Type	Access privileges	Column privileges	Policies
public	data1	table	demo=arwdDxtm/demo+		
			demo2=r*/demo	+	
			demo3=r/demo2		

(1 row)

3.2.15. INSERT 文

INSERT ON CONFLICT 文に DO SELECT 句を指定できるようになりました。
[\[8832709\]](#)

構文

```
INSERT INTO table_name VALUES (...) ON CONFLICT (column_name)
DO SELECT [FOR {UPDATE}|{SHARE}] [RETURNING ...]
```

例 64 INSERT ON CONFLICT DO SELECT 文

```
postgres=> INSERT INTO data1 VALUES (100, 'conflict100') ON CONFLICT (col1)
            DO SELECT FOR UPDATE RETURNING * ;

 col1 |   col2
-----+-----
   100 | conflict100
(1 row)

INSERT 0 1
```

3.2.16. REPACK 文

PostgreSQL 19 に REPACK 文が追加されました。REPACK 文は VACUUM FULL 文と CLUSTER 文を 1 文で実行する SQL 文です。オブジェクトを指定せずに REPACK 文を実行すると、現在のユーザーが MAINTAIN 権限を持つすべてのテーブルとマテリアライズドビューを対象にします。任意のテーブルを指定することができますが、UNLOGGED テーブルやパーティション・テーブルは指定できません。USING INDEX 句を使用すると、指定されたインデックスの情報を元に並べ替えを行います。REPACK 文の進行状況は pg_stat_progress_repack ビューで確認できます。[[ac58465](#), [28d534e](#), [e76d8c7](#), [5dbb63f](#)]

構文

```
REPACK [(options)] [object] [USING INDEX [index_name]]
```

REPACK 文には以下のオプションを指定できます。

表 24 REPACK 文のオプション

オプション	説明
ANALYZE	REPACK 処理完了後に ANALYZE 文を実行
CONCURRENTLY	ロックを最小限にして他の SQL 文のアクセスを許可
VERBOSE	詳細な実行ログを出力

例 65 REPACK 文の実行

```
postgres=> REPACK (ANALYZE, VERBOSE) data1 ;
INFO:  repacking "public.data1" in physical order
INFO:  "public.data1": found 1666667 removable, 3333333 nonremovable row
versions in 27028 pages
DETAIL:  0 dead row versions cannot be removed yet.
CPU: user: 0.58 s, system: 0.20 s, elapsed: 0.94 s.
INFO:  analyzing "public.data1"
INFO:  "data1": scanned 18019 of 18019 pages, containing 3333333 live rows and
0 dead rows; 30000 rows in sample, 3333333 estimated total rows
INFO:  finished analyzing table "postgres.public.data1"
avg read rate: 1582.075 MB/s, avg write rate: 0.088 MB/s
buffer usage: 23 hits, 18023 reads, 1 dirtied
WAL usage: 3 records, 1 full page images, 2165 bytes, 1560 full page image
bytes, 0 buffers full
system usage: CPU: user: 0.06 s, system: 0.01 s, elapsed: 0.08 s
REPACK
```

REPACK 文に CONCURRENTLY オプションを指定すると、テーブルに対して SHARE UPDATE EXCLUSIVE ロックが利用されます。REPACK 文実行中の更新トランザクションは論理デコーディング機能による更新差分が生成されます。更新差分は最終的に新しいテーブルに適用されます。REPACK 文実行中は内部的に「`repack_{ID}`」の名前を持つテンポラリ論理レプリケーションスロットが自動生成されます。REPACK 用のレプリケーションスロットはパラメーター `max_repack_replication_slots` により予約数されます。

3.2.17. SELECT 文

SELECT 文には以下の機能が追加されました。

□ GROUP BY ALL 句

GROUP BY 句に ALL 句を指定できるようになりました。この句を指定すると出力するすべての列を使って GROUP BY 処理を実行します。[[ef38a4d](#)]

例 66 GROUP BY ALL 句の指定

```
postgres=> SELECT col1, col2 FROM data1 GROUP BY ALL ORDER BY 1 ;
 col1 | col2
-----+-----
      1 | data1
      2 | data2
      3 | data3
      4 | data4
(4 rows)
```

□ GROUPING SET とサブクエリー

サブクエリーに GROUPING SETS 列の参照をサポートするようになりました。下記の構文は旧バージョンではエラーになっていました。[[415100a](#)]

例 67 GROUPING SET とサブクエリー

```
postgres=> SELECT (SELECT col1),
                (SELECT col2),
                (SELECT GROUPING(col1, col2)),
                (SELECT SUM(t1.col3)),
                (SELECT max(t2.col3))
FROM
    gstest1 t1 FULL JOIN gstest2 t2 USING (col1, col2)
GROUP BY GROUPING SETS ((col1, col2), ());
```

col1	col2	grouping	sum	max
		3	271	2
1	1	0	147	2
1	3	0	14	
3	4	0	17	
3	3	0	16	
1	2	0	25	2
2	2	0		2
4	1	0	37	
2	3	0	15	

(9 rows)

3.2.18. SET 文

SET 文および ALTER SYSTEM SET 文の値に NULL を指定することで空値を指定できるようになりました。[[ff4597a](#)]

例 68 SET 文に NULL 値指定

```
postgres=> SHOW search_path ;
      search_path
-----
"$user", public
(1 row)

postgres=> SET search_path TO NULL ;
SET
postgres=> SHOW search_path ;
      search_path
-----

(1 row)
```

3.2.19. VACUUM 文

VACUUM VERBOSE 文の出力に以下の情報が追加されるようになりました。

□ メモリー情報

「memory usage:」から開始されるメモリー情報が出力されます。[736f754e]

例 69 メモリー情報

```
postgres=> VACUUM (VERBOSE) data1 ;
INFO:  vacuuming "postgres.public.data1"
...
WAL usage: 185293 records, 92311 full page images, 613464473 bytes, 576198816
full page image bytes, 1416
memory usage: dead item storage 2.58 MB accumulated across 1 reset (limit 64.00
MB each)
system usage: CPU: user: 0.33 s, system: 0.95 s, elapsed: 32.05 s
VACUUM
```

□ パラレル・ワーカー情報

VACUUM 文で使用するパラレル・ワーカーの情報が出力されます。[adcdbe9]

例 70 パラレル・ワーカー情報

```
postgres=> VACUUM (PARALLEL 2, VERBOSE) data1 ;
INFO:  vacuuming "postgres.public.data1"
INFO:  launched 1 parallel vacuum worker for index vacuuming (planned: 2)
INFO:  finished vacuuming "postgres.public.data1": index scans: 1
...
item identifiers removed
parallel workers: index vacuum: 1 planned, 1 launched in total
index "data1_pkey": pages: 27421 in total, 0 newly deleted, 0 currently deleted,
0 reusable
...
system usage: CPU: user: 0.11 s, system: 1.35 s, elapsed: 5.66 s
VACUUM
```

3.2.20. WAIT FOR LSN 文

PostgreSQL 19 に新しい SQL 構文である WAIT FOR LSN 文が追加されました。この SQL 文は主にストリーミング・レプリケーション環境のスタンバイ・インスタンスで実行します。WAIT FOR LSN 文が実行されると指定された LSN が適用されるまで待機します。[\[447aae1\]](#), [3b4e53a](#), [49a181b](#)

構文

```
WAIT FOR LSN 'lsn' [WITH (option [, ...])]
```

オプションには以下を指定できます。

□ MODE '*mode*'

いつまで待機するかを指定します。以下の値を指定できます。primary_flush はプライマリ・サーバで、それ以外はスタンバイ・サーバで指定します。

表 25 MODE オプション設定値

設定値	説明	備考
primary_flush	プライマリ・サーバのフラッシュ	プライマリで実行
standby_replay	スタンバイ・サーバの WAL 適用	
standby_write	スタンバイ・サーバの WAL 書き込み	
standby_flush	スタンバイ・サーバの WAL フラッシュ	

☐ TIMEOUT 'timeout'

最大待機時間を指定します。指定されない場合やゼロ（0）を指定された場合は永久に待機します。

☐ NO_THROW

タイムアウトが発生した場合でもエラーをスローしません。

例 71 WAIT FOR LSN 文の実行

```
postgres=> WAIT FOR LSN '0/0301EF70' ;
status
-----
success
(1 row)

postgres=> WAIT FOR LSN '0/0301FF00' WITH (TIMEOUT '10s') ;
ERROR:  timed out while waiting for target LSN 0/0301FF00 to be replayed:
current standby_replay LSN 0/0301EF70
postgres=> WAIT FOR LSN '0/0301FF00' WITH (TIMEOUT '10s', NO_THROW) ;
status
-----
timeout
(1 row)
```

WAIT FOR LSN 文は以下の値を返します。

表 26 WAIT FOR LSN 文の戻り値

戻り値	説明
success	指定された LSN に到達した
timeout	待機時間がタイムアウトを超過した
not in recovery	実行されたインスタンスがリカバリー状態ではない

3.2.21. 関数

以下の関数が追加／拡張されました。

□ ウィンドウ関数

ウィンドウ関数 LEAD, LAG, FIRST_VALUE, LST_VALUE, NTH_VALUE に IGNORE NULLS 句 (NULL を無視する) と RESPECT NULLS 句 (NULL 値を含む) が指定できるようになりました。デフォルトは RESPECT NULLS です。[[25a30bb](#)]

例 72 IGNORE NULLS 句、RESPECT NULLS 句の指定

```
postgres=> SELECT LEAD(col1) IGNORE NULLS OVER (), LAG(col1) RESPECT NULLS OVER
() FROM data1 ;
 lead | lag
-----+-----
    2 |
    3 |    1
    4 |    2
    5 |    3
    6 |    4
    ...
```

□ encode 関数と decode 関数

エンコーディング方式として base64url と base32hex を指定できるようになりました。[[e1d9171](#), [497c117](#)]

例 73 base64url エンコードと base32hex エンコード

```
postgres=> SELECT encode('¥x69b73eff', 'base64url'),  
                  decode('abc_w', 'base64url') ;
```

encode	decode
abc_w	¥x69b73eff

(1 row)

```
postgres=> SELECT encode('¥x1122334455', 'base32hex'),  
                  decode('24H36H2L', 'base32hex') ;
```

encode	decode
24H36H2L	¥x1122334455

(1 row)

□ error_on_null 関数

入力値が NULL の場合にエラーを発生させ、非 NULL の場合は入力値を返す関数 error_on_null 関数が追加されました。[[2b75c38](#)]

構文

```
anyelement error_on_null (anyelement)
```

例 74 error_on_null 関数の実行

```
postgres=> SELECT error_on_null (NULL::INT) ;  
ERROR:  null value not allowed
```

□ random 関数

最大値と最小値を指定する random 関数のバリエーションに date 型、timestamp 型、timestamp with time zone 型を指定できるようになりました。[[faf071b](#)]

構文

```
date random(min date, max date)
timestamp random(min timestamp, max timestamp)
timestampz random(min timestampz, max timestampz)
```

例 75 random 関数のバリエーション

```
postgres=> SELECT random('2024-09-09'::date, '2025-09-09'::date),
                  random('2024-09-09'::timestampz, '2025-09-09'::timestampz) ;
 random      |                  random
-----+-----
2025-02-04 | 2025-02-16 07:24:48.518771+09
(1 row)
```

□ range_minus_multi 関数、multirange_minus_multi 関数

範囲型から範囲を削除し、複数範囲を返す `range_minus_multi` 関数と `multirange_minus_multi` 関数が追加されました。[[Seed8ce](#)]

構文

```
SETOF anyrange range_minus_multi(anyrange, anyrange)
SETOF anymultirange multirange_minus_multi(anymultirange, anymultirange)
```

例 76 range_minus_multi 関数と multirange_minus_multi 関数

```
postgres=> SELECT range_minus_multi(numrange(1.0, 3.0), numrange(1.5, 2.0)) ;
 range_minus_multi
-----
 [1.0,1.5)
 [2.0,3.0)
(2 rows)

postgres=> SELECT multirange_minus_multi(nummultirange(numrange(1,4)),
      nummultirange(numrange(2,3))) ;
 multirange_minus_multi
-----
 {[1,2),[3,4)}
(1 row)
```

□ tid_block / tid_offset 関数

CTID からブロック番号と、行オフセット番号を取得する tid_block と tid_offset 関数が追加されました。[[df6949c](#)]

構文

```
bigint tid_block(tid)
integer tid_offset(tid)
```

例 77 tid_block / tid_offset 関数の実行

```
postgres=> SELECT ctid, tid_block(ctid), tid_offset(ctid), col1 FROM data1 ;
 ctid | tid_block | tid_offset | col1
-----+-----+-----+-----
(0,1) |         0 |         1 | 100
(0,2) |         0 |         2 | 200
(2 rows)
```

□ width_bucket 関数

width_bucket 関数の最初のパラメーターoperand に NaN を指定できるようになりました。以下の SQL 文は旧バージョンではエラーが発生していました。[[7374b3a](#)]

例 78 width_bucket 関数の実行

```
postgres=> SELECT width_bucket(' NaN', 3.0, 4.0, 888) ;
 width_bucket
-----
            889
(1 row)
```

□ DDL 取得

データベース、テーブル空間、ロールの DDL を出力する関数が追加されました。[[4881981](#), [76e514e](#), [b99fd9f](#), [a4f774c](#)]

構文

```
SETOF text pg_get_database_ddl(regdatabase, VARIADIC text DEFAULT NULL::text)
SETOF text pg_get_tablespace_ddl(name, VARIADIC text DEFAULT NULL::text)
SETOF text pg_get_role_ddl(regrole, VARIADIC text DEFAULT NULL::text)
```

オプションには以下の設定が記述できます。

表 27 オプションの追加

オプション	データ型	説明	備考
pretty	boolean	整形を行うか	
owner	boolean	所有者を含めるか	pg_get_role_ddl には指定不可
tablespace	boolean	表スペースを含めるか	pg_get_database_ddl のみ
memberships	boolean	メンバーを含めるか	pg_get_role_ddl のみ

例 79 データベースの DDL 取得

```
postgres=> SELECT pg_get_database_ddl('postgres', 'pretty', 'true') ;
           pg_get_database_ddl
-----
CREATE DATABASE postgres          +
    WITH TEMPLATE = template0     +
    ENCODING = 'UTF8'             +
    LOCALE_PROVIDER = libc        +
    LOCALE = 'C' ;
ALTER DATABASE postgres OWNER TO postgres;
(2 rows)
```

□ pg_clear_extended_stats 関数

テーブルの拡張統計をクリアする pg_clear_extended_stats 関数が追加されました。

[\[d756fa1\]](#)

構文

```
void pg_clear_extended_stats(schemaname text, relname text,
                             statistics_schemaname text, statistics_name text, inherited boolean)
```

例 80 pg_clear_extended_stats 関数の実行

```
postgres=> CREATE STATISTICS data1_stat1 ON col1,col2 FROM data1 ;
CREATE STATISTICS
postgres=> SELECT pg_clear_extended_stats('public', 'data1', 'public',
                                           'data1_stat1', true) ;
-[ RECORD 1 ]-----+
pg_clear_extended_stats |
```

□ pg_get_multixact_stats 関数

マルチトランザクションの状態を取得する pg_get_multixact_stats 関数が追加されました。この関数は以下の値を返します。 [\[97b1017\]](#)

表 28 pg_get_multixact_stats 関数の戻り値

列名	データ型	説明
num_mxids	bigint	使用中のマルチトランザクション ID
num_members	bigint	使用中のマルチトランザクション ID メンバーのサイズ
members_size	bigint	メンバーが使用しているストレージ・サイズ
oldest_multixact	xid	もっとも古いマルチトランザクション ID

例 81 pg_get_multixact_stats 関数の実行

```
postgres=# SELECT * FROM pg_get_multixact_stats();
 num_mxids | num_members | members_size | oldest_multixact
-----+-----+-----+-----
          1 |           2 |           0 |                1
(1 row)
```

□ pg_get_sequence_data 関数

出力結果に LSN を示す page_lsn 列が追加されました。[b93172c]

例 82 pg_get_sequence_data 関数の実行

```
postgres=> SELECT * FROM pg_get_sequence_data('seq01');
 last_value | is_called | page_lsn
-----+-----+-----
          4 | t         | 3/0701C0A0
(1 row)
```

□ pg_get_publication_tables 関数

pg_get_publication_tables 関数にはテーブル情報を指定するバージョンが追加されました。[fd7a25a]

構文

```
SETOF record pg_get_publication_tables(pubnames text[], target_relid oid)
```

例 83 pg_get_publication_tables 関数の実行

```
postgres=> SELECT * FROM
            pg_get_publication_tables(' {pub1}', 'data1'::regclass::oid) ;
 pubid | relid | attrs | qual
-----+-----+-----+-----
 16405 | 16399 | 1 2   |
(1 row)
```

□ pg_replication_origin_session_setup 関数

オリジンを使用しているワーカースプロセス ID を指定する pid パラメーターが追加されました。省略時の値は 0 で最初のプロセス ID を示します。[[5b14870](#)]
[[5b14870](#)]

例 84 pg_replication_origin_session_setup 関数の定義

```
postgres=> \dfS pg_replication_origin_session_setup
List of functions
-[ RECORD 1 ]-----+-----
Schema          | pg_catalog
Name             | pg_replication_origin_session_setup
Result data type | void
Argument data types | node_name text, pid integer DEFAULT 0
Type             | func
```

□ pg_restore_extended_stats 関数

テーブルの拡張統計をリストアする pg_restore_extended_stats 関数が追加されました。
[[0e80f3f](#), [302879b](#), [ba97bf9](#)]

構文

```
boolean pg_restore_extended_stats(VARIADIC kwargs "any")
```

例 85 pg_restore_extended_stats 関数の実行

```
postgres=> CREATE STATISTICS stat1_data1 ON col1,col2 FROM data1 ;
CREATE STATISTICS
postgres=> SELECT pg_restore_extended_stats(
    'schemaname', 'public',
    'relname', 'data1',
    'statistics_schemaname', 'public',
    'statistics_name', 'stat1_data1',
    'inherited', false,
    'n_distinct', '[{"attributes" : [1,2], "ndistinct" : 4}]'::pg_ndistinct) ;
-[ RECORD 1 ]-----+--
pg_restore_extended_stats | t
```

□ pg_stat_get_backend_wal 関数

pg_stat_get_backend_wal 関数の戻り値に FPI バイトを示す wal_fpi_bytes (numeric 型) が追加されました。[[f9a09aa](#)]

□ pg_sync_replication_slots 関数

pg_sync_replication_slots 関数には同期が完了するまでリトライを行う機能が実装されました。[[0d2d4a0](#)]

3.2.22. JSON

JSON データに関する以下の拡張が実装されました。

□ 追加された jsonpath メソッド

以下の jsonpath メソッドが追加されました。[[bd4f879](#)]

表 29 追加された文字列メソッド

メソッド名	説明
ltrim / rtrim / btrim	前後の文字列を取り除く
lower	小文字変換
upper	大文字変換
initcap	先頭大文字、それ以降を小文字に変換
replace	指定された文字列を変換
split_part	指定された文字列で分割された要素を返す

例 86 追加された jsonpath 文字列メソッドの実行

```

postgres=> SELECT jsonb_path_query('"xTRIMx"'::jsonb, '$.btrim("x")') ;
-[ RECORD 1 ]-----+-----
jsonb_path_query | "TRIM"

postgres=> SELECT jsonb_path_query('"LOWER"'::jsonb, '$.lower()') ;
-[ RECORD 1 ]-----+-----
jsonb_path_query | "lower"

postgres=> SELECT jsonb_path_query('"upper"'::jsonb, '$.upper()') ;
-[ RECORD 1 ]-----+-----
jsonb_path_query | "UPPER"

postgres=> SELECT jsonb_path_query('"iNITCaP"'::jsonb, '$.initcap()') ;
-[ RECORD 1 ]-----+-----
jsonb_path_query | "Initcap"

postgres=> SELECT jsonb_path_query('"REpLACE"'::jsonb, '$.replace("p","P")') ;
-[ RECORD 1 ]-----+-----
jsonb_path_query | "REPLACE"

postgres=> SELECT jsonb_path_query('"test@data"'::jsonb,
                                '$.split_part("@",2)') ;
-[ RECORD 1 ]-----+-----
jsonb_path_query | "data"

```

□ IS JSON 述語

ドメインに対して IS JSON 述語が指定できるようになりました。従来のバージョンでは IS JSON 述語は text 型, json 型, jsonb 型および bytea 型のみ利用可能でした。[[3b4c2b9](#)]

例 87 ドメインに対する IS JSON 述語

```
postgres=> CREATE DOMAIN jd1 AS jsonb CHECK ((VALUE ->'val')::text <> '3') ;
CREATE DOMAIN
postgres=> SELECT ' {"val": 1} '::jd1 IS JSON ;
?column?
-----
t
(1 row)

postgres=> SELECT ' {"val": 3} '::jd1 IS JSON ;
ERROR:  value for domain jd1 violates check constraint "jd1_check"
```

3.2.23. データ型

PostgreSQL 19 のデータ型には以下の拡張が実装されました。

□ regdatabase 型

データベースの OID を示すオブジェクト識別子データ型 regdatabase が追加されました。カレントのデータベースの oid を取得する場合には以下のように書けます。[[bd09f02](#)]

例 88 regdatabase データ型による検索

```
postgres=> SELECT * FROM pg_shdepend WHERE
           dbid = current_database()::regdatabase ;
 dbid | classid | objid | objsubid | refclassid | refobjid | deptype
-----+-----+-----+-----+-----+-----+-----
    5 |    2615 |   2200 |         0 |         1260 |    16384 | a
(1 row)
```

□ oid8 型

8 バイト OID を示す oid8 型が追加されました。[[b139bd3](#)]

例 89 oid8 データ型

```
postgres=> SELECT 999999999::oid8 ;
      oid8
-----
 999999999
(1 row)
```

□ uuid 型と bytea 型の変換

uuid 型と bytea 型が相互変換できるようになりました。[[ba21f5b](#)]

例 90 uuid 型と bytea 型のデータ型変換

```
postgres=> SELECT '¥x019a2f859ced7225b99d9c55044a2563'::bytea::uuid ;
      uuid
-----
 019a2f85-9ced-7225-b99d-9c55044a2563
(1 row)

postgres=> SELECT '5b35380a-7143-4912-9b55-f322699c6770'::uuid::bytea ;
      bytea
-----
 ¥x5b35380a714349129b55f322699c6770
(1 row)
```

□ inet 型と cidr 型による GiST インデックス

inet 型と cidr 型の GiST インデックスの演算子クラスがそれぞれ inet_ops と cidr_ops に変更されました。これにより btree_gist エクステンションの導入が必要なくなりました。以下の CREATE INDEX 文は PostgreSQL 18 以前ではエラーになります。pg_upgrade コマンドは旧バージョンのデータベースに gist_inet_ops または gist_cidr_ops 演算子クラスが含まれる場合にはエラーが発生します。[[b352d3d](#)]

例 91 inet 型と cidr 型に対する GiST インデックス

```
postgres=> CREATE TABLE data1(col1 INET, col2 CIDR) ;
CREATE TABLE
postgres=> CREATE INDEX idx1_data1 ON data1 USING gist (col1) ;
CREATE INDEX
postgres=> CREATE INDEX idx2_data1 ON data1 USING gist (col2) ;
CREATE INDEX
```

□ tsearch 関連データ型

解析にデータベースのデフォルト照合順序を使うようになりました。以前のバージョンではデータベースの CTYPE 設定を使っていました。[[e113f9c](#)]

3.2.24. PL/pgSQL

PL/pgSQL には以下の機能が追加されました。

□ シンプルな SELECT INTO 文の変換

単一変数に代入する単純なスカラー式を使った SELECT INTO 文の実行は SQL 文の実行ではなく、変数への代入処理に変換されるようになりました。この修正により、対象の SQL 文は pg_stat_statements モジュールへのフックが実行されなくなります。[[ce8d5fe](#)]

□ 予約語の削除

予約語リストから EXECUTE と STRICT が削除されました。これらの単語を変数名等に変更できるようになりました。[[999f172](#)]

例 92 旧予約語を変数名に使用

```
postgres=> DO $$
DECLARE
EXECUTE INT;
BEGIN
EXECUTE := 100;
END;
$$;
DO
postgres=>
```


3.2.25. PL/Python

イベントトリガーのサポートが追加されました。[53eff47]

3.2.26. オプティマイザー

より高速な実行計画が選択されるようになりました。

□ 積極的集約

結合処理の入力タプル数を削減するために結合前に部分的な集約を行う積極的集約最適化が実装されました。この最適化はパラメーター`enable_eager_aggregate`で制御されます。デフォルト値は`on`で、有効になっています。積極的集約を検討するために必要なグループサイズはパラメーター`min_eager_agg_group_size`で指定します。このパラメーターのデフォルト値は8です。[8e11859]

例 93 PostgreSQL 18 の実行計画

```
postgres=> EXPLAIN (VERBOSE, COSTS OFF) SELECT t1.col1, avg(t2.col3)
           FROM eager1 AS t1 JOIN eager2 AS t2 ON t1.col2 = t2.col2
           GROUP BY t1.col1 ORDER BY t1.col1 ;
           QUERY PLAN
```

GroupAggregate

Output: t1.col1, avg(t2.col3)

Group Key: t1.col1

-> Sort

Output: t1.col1, t2.col3

Sort Key: t1.col1

-> Hash Join

Output: t1.col1, t2.col3

Hash Cond: (t2.col2 = t1.col2)

-> Seq Scan on public.eager2 t2

Output: t2.col1, t2.col2, t2.col3

-> Hash

Output: t1.col1, t1.col2

-> Seq Scan on public.eager1 t1

Output: t1.col1, t1.col2

(15 rows)

例 94 PostgreSQL 19 の実行計画

```
postgres=> EXPLAIN (VERBOSE, COSTS OFF) SELECT t1.col1, avg(t2.col3)
           FROM eager1 AS t1 JOIN eager2 AS t2 ON t1.col2 = t2.col2
           GROUP BY t1.col1 ORDER BY t1.col1 ;
           QUERY PLAN
```

Finalize GroupAggregate

Output: t1.col1, avg(t2.col3)

Group Key: t1.col1

-> Sort

Output: t1.col1, (PARTIAL avg(t2.col3))

Sort Key: t1.col1

-> Hash Join

Output: t1.col1, (PARTIAL avg(t2.col3))

Hash Cond: (t1.col2 = t2.col2)

-> Seq Scan on public.eager1 t1

Output: t1.col1, t1.col2, t1.col3

-> Hash

Output: t2.col2, (PARTIAL avg(t2.col3))

-> Partial HashAggregate

Output: t2.col2, PARTIAL avg(t2.col3)

Group Key: t2.col2

-> Seq Scan on public.eager2 t2

Output: t2.col1, t2.col2, t2.col3

(18 rows)

□ ANTI JOIN で Memoize 使用

SEMI JOIN または ANTI JOIN で Memoize 処理が利用できるようになりました。

[\[Oda29e4\]](#)

例 95 PostgreSQL 18 の実行計画

```
postgres=> EXPLAIN (COSTS OFF, BUFFERS OFF)
SELECT COUNT(*) FROM data1 d1 LEFT JOIN
LATERAL (SELECT DISTINCT ON (c1) c1, c2, d1.c1 AS x FROM data1 d2) d2
ON d1.c1 + 1 = d2.c1 WHERE d2.c1 IS NULL ;
```

QUERY PLAN

Aggregate

- > Nested Loop Anti Join
 - > Seq Scan on data1 d1
 - > Subquery Scan on d2
 - Filter: ((d1.c1 + 1) = d2.c1)
 - > Unique
 - > Sort
 - Sort Key: d2_1.c1
 - > Seq Scan on data1 d2_1

(9 rows)

例 96 PostgreSQL 19 の実行計画

```
postgres=> EXPLAIN (COSTS OFF, BUFFERS OFF)
SELECT COUNT(*) FROM data1 d1 LEFT JOIN
LATERAL (SELECT DISTINCT ON (c1) c1, c2, d1.c1 AS x FROM data1 d2) d2
ON d1.c1 + 1 = d2.c1 WHERE d2.c1 IS NULL ;
          QUERY PLAN
```

Aggregate

```
-> Nested Loop Anti Join
    -> Seq Scan on data1 d1
        -> Memoize
            Cache Key: (d1.c1 + 1), d1.c1
            Cache Mode: binary
        -> Subquery Scan on d2
            Filter: ((d1.c1 + 1) = d2.c1)
                -> Unique
                    -> Sort
                        Sort Key: d2_1.c1
                    -> Seq Scan on data1 d2_1
```

(12 rows)

□ COUNT 関数の書き換え

書き換え可能な場合は `COUNT(any)` 句を `COUNT(*)` 句に書き換えます。書き換えは非 `NULL` 列が指定された場合や `ORDER BY` 句、`DISTINCT` 句が無い場合に限られます。この変更によりオーバーヘッドを削減し、パフォーマンスを向上させます。[[42473b3](#)]

例 97 PostgreSQL 18 の実行計画

```
postgres=> EXPLAIN (VERBOSE) SELECT COUNT(col1) FROM data1 ;  
               QUERY PLAN  
  
-----  
Aggregate  (cost=17906.00..17906.01 rows=1 width=8)  
  Output: count(col1)  
    -> Seq Scan on public.data1  (cost=0.00..15406.00 rows=1000000 width=4)  
        Output: col1, col2  
(4 rows)
```

例 98 PostgreSQL 19 の実行計画

```
postgres=> EXPLAIN (VERBOSE) SELECT COUNT(col1) FROM data1 ;  
               QUERY PLAN  
  
-----  
Aggregate  (cost=17906.00..17906.01 rows=1 width=8)  
  Output: count(*)  
    -> Seq Scan on public.data1  (cost=0.00..15406.00 rows=1000000 width=0)  
        Output: col1, col2  
(4 rows)
```

☐ TID Range Scan の並列化

TID Range Scan をパラレルクエリーで実行できるようになりました。[[Oca3b16](#)]

例 99 PostgreSQL 18 の実行計画

```
postgres=> EXPLAIN SELECT * FROM data1 WHERE ctid > ' (10,0)' AND ctid < ' (30,1)';  
               QUERY PLAN  
  
-----  
Tid Range Scan on data1  (cost=0.01..61.00 rows=3700 width=10)  
  TID Cond: ((ctid > ' (10,0)'::tid) AND (ctid < ' (30,1)'::tid))  
(2 rows)
```

例 100 PostgreSQL 19 の実行計画

```
postgres=> EXPLAIN SELECT * FROM data1 WHERE ctid > ' (10,0)' AND ctid < ' (30,1)';
               QUERY PLAN
-----
Gather  (cost=0.01..33.25 rows=3700 width=10)
  Workers Planned: 4
    -> Parallel Tid Range Scan on data1  (cost=0.01..33.25 rows=925 width=10)
          TID Cond: ((ctid > ' (10,0)::tid) AND (ctid < ' (30,1)::tid))
(4 rows)
```

□ 部分 HASH インデックス

部分インデックスとして作成された HASH インデックスが条件に合致する WHERE 句を指定された場合、実行計画に使用できるようになりました。この仕様は PostgreSQL 14 以降にバックポートされます。[[5528e8d](#)]

例 101 PostgreSQL 18 の実行計画

```
postgres=> CREATE INDEX idx1_data1 ON data1 USING hash (col2) WHERE col2=1000 ;
CREATE INDEX
postgres=> EXPLAIN SELECT * FROM data1 WHERE col2=2000 ;
               QUERY PLAN
-----
Seq Scan on data1  (cost=0.00..17906.00 rows=1 width=14)
  Filter: (col2 = 2000)
(2 rows)
```

例 102 PostgreSQL 19 の実行計画

```
postgres=> CREATE INDEX idx1_data1 ON data1 USING hash (col2) WHERE col2=1000 ;
CREATE INDEX
postgres=> EXPLAIN SELECT * FROM data1 WHERE col2=1000 ;
               QUERY PLAN
-----
Index Scan using idx1_data1 on data1  (cost=0.00..8.02 rows=1 width=14)
  Index Cond: (col2 = 1000)
(2 rows)
```

□ IS DISTINCT FROM 句の最適化

NOT NULL 制約列に対する IS DISTINCT FROM 句は「<>」演算子に変換されるようになりました。[\[0a37961\]](#)

例 103 PostgreSQL 18 の実行計画

```
postgres=> EXPLAIN SELECT * FROM data1 WHERE col1 IS DISTINCT FROM col1 ;
               QUERY PLAN
-----
Seq Scan on data1  (cost=0.00..17906.00 rows=995000 width=10)
  Filter: (col1 IS DISTINCT FROM col1)
(2 rows)
```

例 104 PostgreSQL 19 の実行計画

```
postgres=> EXPLAIN SELECT * FROM data1 WHERE col1 IS DISTINCT FROM col1 ;
               QUERY PLAN
-----
Seq Scan on data1  (cost=0.00..17906.00 rows=995000 width=10)
  Filter: (col1 <> col1)
(2 rows)
```

□ NOT IN 句の Anti Join 変換

安全であると確認される場合、NOT IN 句は実行計画として Anti Join を使用します。
[\[383eb21\]](#)

例 105 PostgreSQL 18 の実行計画

```
postgres=> EXPLAIN SELECT * FROM data1 WHERE col1 NOT IN (SELECT col1 FROM
              data1 WHERE col1 < 10000) ;
              QUERY PLAN

-----
Seq Scan on data1 (cost=194.99..364.97 rows=5000 width=8)
  Filter: (NOT (ANY (col1 = (hashed SubPlan 1).col1)))
  SubPlan 1
    -> Seq Scan on data1 data1_1 (cost=0.00..169.99 rows=9999 width=4)
        Filter: (col1 < 10000)
(5 rows)
```

例 106 PostgreSQL 19 の実行計画

```
postgres=> EXPLAIN SELECT * FROM data1 WHERE col1 NOT IN (SELECT col1 FROM
              data1 WHERE col1 < 10000) ;
              QUERY PLAN

-----
Hash Anti Join (cost=29.98..48.71 rows=1 width=8)
  Hash Cond: (data1.col1 = data1_1.col1)
  -> Seq Scan on data1 (cost=0.00..14.99 rows=999 width=8)
  -> Hash (cost=17.49..17.49 rows=999 width=4)
      -> Seq Scan on data1 data1_1 (cost=0.00..17.49 rows=999 width=4)
          Filter: (col1 < 10000)
(6 rows)
```

□ Result ノードの詳細

Result ノードに結果出力元の情報が追加されました。これにより EXPLAIN 文の実行結果に「Replace:」ノードの情報が追加されるようになります。[\[f2bae51\]](#)

例 107 PostgreSQL 19 の実行計画

```
postgres=> EXPLAIN (COSTS OFF) SELECT * FROM data1 WHERE
```

```
col1=(SELECT MAX(col1) FROM data1) ;
```

```
QUERY PLAN
```

```
-----  
Index Scan using data1_pkey on data1
```

```
Index Cond: (col1 = (InitPlan expr_1).col1)
```

```
InitPlan expr_1
```

```
-> Result
```

```
Replaces: MinMaxAggregate
```

```
InitPlan minmax_1
```

```
-> Limit
```

```
-> Index Only Scan Backward using data1_pkey on data1 ...
```

```
(8 rows)
```

3.3. パラメーターの変更

PostgreSQL 18 では以下のパラメーターが変更されました。

3.3.1. 追加されたパラメーター

以下のパラメーターが追加されました。パラメーター `file_extend_method` は PostgreSQL 16 以降にバックポートされます。[[06473f5](#), [8e11859](#), [dd3ae37](#), [67c2097](#), [f94e914](#), [1d92e0c](#), [4f43302](#), [d7965d6](#), [a8f45de](#), [1ff3180](#), [e76d8c7](#), [d1c01b7](#)]

表 30 追加されたパラメーター

パラメーター	説明 (context)	デフォルト値
<code>autovacuum_analyze_score_weight</code>	自動 VACUUM の ANALYZE 優先順位 ウェイト (sighup)	1.0
<code>autovacuum_freeze_score_weight</code>	自動 VACUUM の FREEZE 優先順位 ウェイト (sighup)	1.0
<code>autovacuum_max_parallel_workers</code>	自動 VACUUM の最大パラレル・ワーカー数 (sighup)	0
<code>autovacuum_multixact_freeze_score_weight</code>	自動 VACUUM マルチトランザクション FREEZE 優先順位 ウェイト (sighup)	1.0
<code>autovacuum_vacuum_insert_score_weight</code>	自動 VACUUM の INSERT の VACUUM 優先順位 ウェイト (sighup)	1.0
<code>autovacuum_vacuum_score_weight</code>	自動 VACUUM の VACUUM 優先順位 ウェイト (sighup)	1.0
<code>autovacuum_worker_slots</code>	VACUUM 処理のバックエンド・スロット数 (postmaster)	16
<code>debug_exec_backend</code>	稼働サーバーが EXEC_BACKEND を有効にして起動しているか (internal)	off
<code>debug_print_raw_parse</code>	パースツリーを変換せずに出力する (user)	off
<code>effective_wal_level</code>	有効な WAL 出力レベル (internal)	replica
<code>enable_eager_aggregate</code>	積極アグリゲーション最適化を利用する (user)	on

パラメーター	説明 (context)	デフォルト値
file_extend_method	ファイルの拡張メソッド (sighup)	posix_fallocate
hosts_file	SSL/SNI ファイル名 (postmaster)	{PGDATA}/pg_hosts.conf
io_max_workers	I/O ワーカーの最大数 (sighup)	8
io_min_workers	I/O ワーカーの最小数 (sighup)	2
io_worker_idle_timeout	I/O ワーカーの最大アイドル時間 (sighup)	60000
io_worker_launch_interval	新規 I/O ワーカーを起動する最小間隔 (sighup)	100ms
log_autoanalyze_min_duration	自動 ANALYZE 実行ログ出力時間 (sighup)	10min
max_repack_replication_slots	REPACK 文実行用に予約されたレプリケーションスロット数 (postmaster)	5
min_eager_agg_group_size	積極アグリゲーションを利用する際最小メモリ量 (user)	8
password_expiration_warning_threshold	パスワード期限切れの警告を出力する期間 (sighup)	7d
ssl_sni	SSL 接続で SNI 機能を使用するか (sighup)	off
timing_clock_source	タイミング測定に使用するクロックソースを指定 (superuser)	auto (tsc)
wal_sender_shutdown_timeout	wal sender プロセスの停止タイムアウト (user)	-1

□ パスワード期限切れの警告開始

パスワードの有効期限が password_expiration_warning_threshold パラメーター (デフォルト値 7d) 以内になった場合、認証時に警告が出力されます。[1d92e0c]

例 108 パスワード期限切れ警告

```
$ psql -h dbsvr1 -d postgres -U user1
Password for user user1: <<Password>>
WARNING:  role password will expire soon
DETAIL:  The password for role "user1" will expire in 13 hours.
psql (19beta1)
Type "help" for help.

postgres=>
```

3.3.2. 変更されたパラメーター

以下のパラメーターは設定範囲や選択肢が変更されました。[[38e0190](#), [1efdd7c](#), [8a6af3a](#), [12da457](#), [bf5da5d](#)]

表 31 変更されたパラメーター

パラメーター	変更内容
data_checksums	チェックサム有効化途中の場合は <code>inprogress-on</code> に変更される。データ型が <code>boolean</code> から <code>enum</code> に変更
extension_control_path	指定されたパス内に同じ名前のエクステンションが存在した場合、 <code>pg_available_extensions</code> ビューに最初に見つかったパスを出力する
io_workers	<code>io_max_workers</code> と <code>io_min_workers</code> に分割
log_min_messages	プロセスの種類毎にログ出力レベルを変更可能に
sync_replication_slots	有効にするにはパラメーター <code>wal_level</code> を <code>logical</code> に変更が必要
wal_receiver_timeout	設定値変更のコンテキストが <code>sighup</code> から <code>user</code> に変更

□ パラメーター `log_min_messages` の指定

`log_min_messages` パラメーターはプロセス毎にレベルを変更できるようになりました。プロセス名とレベルをコロン (:) で区切り、複数のプロセス指定はカンマ (,) で区切って指定します。[[38e0190](#)]

表 32 プロセスの指定文字列

プロセス名指定	説明
archiver	WAL アーカイバ
autovacuum	自動 VACUUM ラウンチャ、自動 VACUUM ワーカー
backend	バックエンド・プロセス
bgworker	バックグラウンド・ワーカー
bgwriter	バックグラウンド・ライター
checkpointer	チェックポインター
ioworker	I/O ワーカー
postmaster	ポストマスター・プロセス
slotsyncworker	レプリケーションスロット同期ワーカー
startup	スタートアップ・プロセス
syslogger	ロガー・プロセス
walreceiver	WAL 受信プロセス
walsender	WAL 送信プロセス
walsummarizer	WAL サマライザ・プロセス
walwriter	WAL ライター・プロセス

例 109 パラメーターlog_min_messages の変更

```
postgres=# SET log_min_messages TO 'warning, autovacuum:debug1' ;
SET
```

3.3.3. デフォルト値が変更されたパラメーター

以下のパラメーターはデフォルト値が変更されました。[[2aac62b](#), [7f8c88c](#), [79534f9](#)]

表 33 デフォルト値が変更されたパラメーター

パラメーター	PostgreSQL 18	PostgreSQL 19	備考
jit	on	off	
log_lock_waits	off	on	
max_locks_per_transaction	64	128	
server_version	18.4	19beta1	
server_version_num	180004	190000	

`configure` コマンドで `--with-lz4` を指定して作成されたバイナリでは `default_toast_compression` パラメーターのデフォルト値は `lz4` になります。

3.3.4. 削除されたパラメーター

以下のパラメーターは削除されました。[[4576208](#)]

表 34 削除されたパラメーター

パラメーター	理由
<code>escape_string_warning</code>	<code>standard_conforming_strings</code> パラメーターの固定により不要になったため

3.4. ユーティリティ

ユーティリティ・コマンドの主な機能拡張点を説明します。

3.4.1. oid2name

--extended オプションを指定した場合にファイルのパスを示す「Path」項目が出力されるようになりました。[[3c5ec35](#)]

例 110 oid2name コマンドの実行

```
$ oid2name --table=data1 --extended
From database "postgres":
  Filenode  Table Name    Oid  Schema  Tablespace      Path
-----
    16385      data1  16385  public  pg_default  base/5/16385
```

3.4.2. pg_archivecleanup

dry-run モードを指定したことを示すログが出力されるようになりました。この仕様は pg_createsubscriber コマンド、pg_combinebackup コマンドにも追加されました。[[c05dee1](#)]

例 111 dry-run モード

```
$ pg_archivecleanup --dry-run arch 000000010000000000000000A.00000028.backup
pg_archivecleanup: Executing in dry-run mode.
No files will be removed.
arch/00000001000000000000000001
arch/00000001000000000000000002
arch/00000001000000000000000003
...
```

3.4.3. pg_createsubscriber

pg_createsubscriber コマンドには以下の機能が追加されました。

□ ログ出力用ディレクトリ

`pg_createsubscriber` コマンドにログ出力用ディレクトリを示す `--logdir` オプション（短縮形 `-l`）が追加されました。[6b5b7ea]

例 112 ログ用ディレクトリの指定

```
$ pg_createsubscriber -d postgres -P "dbname=postgres user=postgres port=5432"
-l ~/log --publication pub1 --subscription sub1 -D data.s
$ ls ~/log/
20260604T174538.189  20260604T174611.016
$ ls ~/log/20260604T174538.189/pg_createsubscriber_server.log
/home/postgres/log/20260604T174538.189/pg_createsubscriber_server.log
$ cat ~/log/20260604T174538.189/pg_createsubscriber_server.log
LOG:  redirecting log output to logging collector process
HINT:  Future log output will appear in directory "log".
```

□ レプリケーションスロットの再利用

指定されたレプリケーションスロットが存在する場合には再利用されるようになりました。[85ddcc2]

□ リカバリー用パラメーター設定ファイル

`pg_createsubscriber` コマンド実行時に一時的に設定されるリカバリー用のパラメーターが `pg_createsubscriber.conf` に設定されます。旧バージョンでは `postgresql.auto.conf` ファイルに追記されていました。`pg_createsubscriber.conf` ファイルを有効化するために `postgresql.auto.conf` ファイルに「`include_if_exists 'pg_createsubscriber.conf'`」が追加されます。リカバリー処理完了後に `pg_createsubscriber.conf` ファイルはファイル名が `pg_createsubscriber.conf.disabled` に変更されます。[639352d]

例 113 pg_createsubscriber コマンド実行後のパラメーターファイル

```
$ pg_createsubscriber -d postgres -p 5433 -P "host=dbsvr1 port=5432
dbname=postgres user=postgres" -D data.s --publication=pub1
--subscription=sub1
LOG: redirecting log output to logging collector process
HINT: Future log output will appear in directory "log".
LOG: redirecting log output to logging collector process
HINT: Future log output will appear in directory "log".
$ tail -1 data.s/postgresql.auto.conf
include_if_exists 'pg_createsubscriber.conf'
$ cat data.s/pg_createsubscriber.conf.disabled
primary_conninfo      = 'user=postgres  passfile='' /home/postgres/.pgpass''
channel_binding=prefer  port=5432  sslmode=prefer  sslnegotiation=postgres
sslcompression=0  sslcertmode=allow  sslsni=1  ssl_min_protocol_version=TLSv1.2
gssencmode=disable  krbsrvname=postgres  gssdelegation=0  target_session_attrs=any
load_balance_hosts=disable'
recovery_target = ''
recovery_target_timeline = 'latest'
recovery_target_inclusive = false
recovery_target_action = promote
recovery_target_name = ''
recovery_target_time = ''
recovery_target_xid = ''
recovery_target_lsn = '0/05000320'
```

3.4.4. pg_dump

pg_dump コマンドには以下の機能が追加されました。

☐ 拡張統計情報のダンプ

拡張統計情報が pg_restore_extended_stats 関数を使って出力されるようになりました。

[\[c32fb29\]](#)

例 114 拡張統計情報のダンプ

```
$ pg_dump -d postgres -U demo --statistics-only | grep restore
SELECT * FROM pg_catalog.pg_restore_relation_stats(
SELECT * FROM pg_catalog.pg_restore_attribute_stats(
SELECT * FROM pg_catalog.pg_restore_attribute_stats(
SELECT * FROM pg_catalog.pg_restore_attribute_stats(
SELECT * FROM pg_catalog.pg_restore_extended_stats(
```

3.4.5. pg_dumpall

`pg_dumpall` コマンドはテキスト形式以外の出力フォーマットをサポートするようになりました。出力フォーマットは `--format` オプション（短縮形 `-F`）で指定します。デフォルト値は従来通りテキスト（`--format=p`）です。フォーマットには以下の値を指定できます。テキスト形式の場合以外は出力先（`--file` オプション）にはディレクトリ名を指定します。`pg_dumpall` コマンドで出力したデータをリストアする場合、テキスト形式を指定した場合以外は `pg_restore` コマンドを使用します。[763aaa0]

表 35 フォーマット指定値

フォーマット指定	説明	備考
p	テキスト形式	デフォルト
c	カスタム・ダンプ形式（拡張子.dmp）	
t	tar 形式（拡張子.tar）	
d	テーブル単位に gzip 形式（拡張子.gz）	

例 115 gzip 形式でダンプ

```
$ pg_dumpall --format=d --file=dump.d
$ ls -l dump.d/databases/5/
total 2316
-rw-r--r--. 1 postgres postgres 2362652 Feb 27 14:46 4395.dat.gz
-rw-r--r--. 1 postgres postgres    25 Feb 27 14:46 4396.dat.gz
-rw-r--r--. 1 postgres postgres  2568 Feb 27 14:46 toc.dat
$
```

3.4.6. pg_restore

pg_restore コマンドには以下のオプションが追加されました。

□ --no-globals オプション

グローバル・オブジェクト（ロール、表スペース）のリストアを行わないオプション--no-globals オプションが追加されました。バックアップ用データとしてテキスト形式以外の pg_dumpall コマンド出力ファイルが必要です。[[3c19983](#)]

例 116 --no-globals オプションの指定

```
$ pg_restore -C --no-globals --format=d dump.d -d postgres -v
pg_restore: skipping restore of global objects because --no-globals was
specified
pg_restore: found database "template1" (OID: 1) in file "dump.d/map.dat"
...
pg_restore: executing SELECT pg_catalog.set_config('search_path', '', false);
pg_restore: creating TABLE "public.data1"
pg_restore: processing data for table "public.data1"
pg_restore: creating CONSTRAINT "public.data1_data1_pkey"
pg_restore: creating ACL "SCHEMA public"
pg_restore: number of restored databases is 2
```

□ --globals-only オプション

pg_dumpall コマンドに対応して、グローバル・データのみリストアする--globals-only オプションが追加されました。[[763aaa0](#)]

3.4.7. pg_upgrade

pg_upgrade コマンドには以下の機能が追加されました。

□ インプレース・テーブル空間の移行

インプレース・テーブル空間は新バージョンのデータベース・クラスタに移行されるようになりました。旧バージョンではインプレース・テーブル空間はコピーされませんでした。この機能を使用する場合、移行先のデータベースではあらかじめ allow_in_place_tablespaces パラメーターを on に設定しておきます。インプレース・テーブル空間の有無は pg_upgrade --check コマンドでは検証されません。[[3bcfd8](#)]

□ ラージオブジェクト移行の高速化

`pg_upgrade` コマンドは `pg_largeobject_metadata` カタログと `pg_shdepend` カタログの移行時に `COPY FROM stdin` 文を実行してデータの一括移行を行います。以前のバージョンではラージオブジェクト毎に `lo_create` 関数、`ALTER LARGE OBJECT` 文、`GRANT` 文を実行していました。[[b33f753](#)]

3.4.8. `pg_verifybackup`

`tar` フォーマットで作成されたバックアップに対して `pg_verifybackup` コマンドが実行できるようになりました。旧バージョンでは WAL ファイルのパースを行わない (`--no-parse-wal`) オプションが必要でした。[[b3cf461](#), [f8a0cd2](#), [b15c151](#)]

例 117 `tar` フォーマットのバックアップに対する `pg_verifybackup` コマンド

```
$ pg_basebackup --format=t --pgdata=back
$ pg_verifybackup --format=t --progress back
24984/24984 kB (100%) verified
backup successfully verified
```

3.4.9. `pg_waldump`

`tar` フォーマットで作成されたバックアップに対して `pg_waldump` コマンドが実行できるようになりました。[[b15c151](#)]

例 118 `tar` フォーマットのバックアップに対する `pg_waldump` コマンド

```
$ pg_basebackup --format=t -D back
$ pg_waldump back/pg_wal.tar --start=0/08000028 --end=0/08000088
rmgr: XLOG          len (rec/tot):   34/   34, tx:          0, lsn: 0/08000...
rmgr: Standby       len (rec/tot):   54/   54, tx:          0, lsn: 0/08000...
```

3.4.10. `psql`

`psql` コマンドには以下の新機能が実装されました。

□ SERVICEFILE 変数

接続に使用したサービスファイルを示す変数 `SERVICEFILE` が追加されました。

[\[6b1c4d3\]](#)

例 119 SERVICEFILE 変数の表示

```
$ psql "servicefile=/tmp/serviceA.conf service=service1"
psql (19beta1)
Type "help" for help.

postgres=# \echo :SERVICEFILE
/tmp/serviceA.conf
```

□ %S 置換変数

`psql` コマンドの入力プロンプトを示す `PROMPT1`、`PROMPT2`、`PROMPT3` に置換変数 `%S` を指定できるようになりました。この変数はパラメーター `search_path` の値に変換されます。 [\[b3ce55f\]](#)

例 120 PROMPT1 変数の変更

```
postgres=> SHOW search_path ;
-[ RECORD 1 ]-----
search_path | "$user", public

postgres=> \set PROMPT1 ' [%S]=> '
["$user", public]=> SET search_path = 'public' ;
SET
[public]=>
```

□ %i 置換変数

`psql` コマンドの入力プロンプトを示す `PROMPT1`、`PROMPT2`、`PROMPT3` に置換変数 `%i` を指定できるようになりました。この変数はパラメーター `in_hot_standby` を元に置換されます。 [\[dddbbc2\]](#)

表 36 %i 置換変数の設定

in_hot_standby 設定値	%i 置換文字列	備考
on	primary	
off	standby	
-	?	in_hot_standby が無いバージョン

例 121 PROMPT1 変数の変更

```
postgres=> SHOW in_hot_standby ;
-[ RECORD 1 ]--+-
in_hot_standby | off

postgres=> \set PROMPT1 '[%i]=> '
[primary]=>
```

□ BOOLEAN 値の出力

`\set` コマンドにより BOOLEAN 値の出力文字を指定できるようになりました。変数 `display_true` と `display_false` を変更します。この変数のデフォルト値は「f」(false の場合) と「t」(true の場合) です。[[645cb44](#)]

例 122 BOOLEAN 値の出力

```
postgres=> \set display_true 'True'
Boolean true display is "True".
postgres=> \set display_false 'False'
Boolean false display is "False".
postgres=> SELECT true::boolean, false::boolean ;
  bool | bool
-----+-----
  True | False
(1 row)
```

□ コメントの出力

パブリケーション、拡張統計、サブスクリプションに追加されたコメントが、メタコマンド `\drp+`、`\drs+` および `\dx+` の出力に含まれるようになりました。[[aecc558](#)]

例 123 コメントの出力

```
postgres=> CREATE PUBLICATION pub1 FOR TABLE data1 ;
CREATE PUBLICATION
postgres=> COMMENT ON PUBLICATION pub1 IS 'comment for pub1' ;
COMMENT
postgres=> \dRp+
```

Publication pub1						
Owner	All tables	All sequences	Inserts	...	Via root	<u>Description</u>
demo	f	f	t	...	f	comment for pub1

Tables:

 "public.data1"

□ パブリケーション情報

テーブル定義の出力にパブリケーションに含む場合と、含まれない場合の情報が出力されるようになりました。[[2e1d4fd](#)]

例 124 テーブルのパブリケーション対応情報

```
postgres=# CREATE TABLE data1(col1 INT PRIMARY KEY, col2 VARCHAR(10)) ;
CREATE TABLE
postgres=# CREATE PUBLICATION pub1 FOR TABLE data1 ;
CREATE PUBLICATION
postgres=# CREATE PUBLICATION pub2 FOR ALL TABLES EXCEPT (TABLE data1) ;
CREATE PUBLICATION
postgres=# \d data1
```

Table "public.data1"				
Column	Type	Collation	Nullable	Default
col1	integer		not null	
col2	character varying(10)			

Indexes:

"data1_pkey" PRIMARY KEY, btree (col1)

Included in publications:

"pub1"

Excluded from publications:

"pub2"

3.4.11. vacuumdb

vacuumdb コマンドには以下のオプションが追加されました。

□ --dry-run オプションの追加

実行する SQL 文を表示だけ行う --dry-run オプションが追加されました。[[d107176](#)]

例 125 --dry-run オプションの指定

```
$ vacuumdb -d postgres --dry-run --schema=public
vacuumdb: Executing in dry-run mode.
No commands will be sent to the server.
vacuumdb: vacuuming database "postgres"
VACUUM (SKIP_DATABASE_STATS) public.data1;
VACUUM (SKIP_DATABASE_STATS) public.data2;
VACUUM (SKIP_DATABASE_STATS) public.data3;
VACUUM (ONLY_DATABASE_STATS);
$
```

□ --analyze-only オプションの動作変更

`vacuumdb` コマンドに `--analyze-only` オプションを指定した場合、パーティション・テーブルも統計情報を取得するようになりました。以前のバージョンではパーティション・テーブルは統計情報取得の対象外でした。[[6429e5b](#)]

3.5. Contrib モジュール

Contrib モジュールに関する新機能を説明しています。

3.5.1. auto_explain

auto_explain モジュールには以下のパラメーターが追加されました。

□ log_extension_options パラメーター

パラメーターauto_explain.log_extension_options が追加されました。このパラメーターにはpg_overexplainのような追加モジュールのEXPLAIN文に渡される文字列を指定します。

例 126 log_extension_options オプションの指定

```
postgres=# LOAD 'auto_explain' ;
LOAD
postgres=# LOAD 'pg_overexplain' ;
LOAD
postgres=# SET auto_explain.log_extension_options = 'DEBUG' ;
SET
```

□ log_io パラメーター

パラメーターauto_explain.log_io が追加されました。デフォルト値はoffです。このパラメーターをonに設定すると、ログにSQL文が実行したI/O統計が出力されます。この機能はパラメーターauto_explain.log_analyzeをonに設定した場合のみ効果があります。

[\[61c36a3\]](#)

□ log_extension_options パラメーター

pg_overexplain モジュールや、pg_plan_advice モジュール等の外部モジュールの追加オプションをログに含めます。

例 127 log_io オプションの指定

```
postgres=# SET auto_explain.log_analyze = on ;
SET
postgres=# SET auto_explain.log_io = on ;
SET
postgres=# SELECT COUNT(*) FROM data1 ;
count
-----
10000000
(1 row)

postgres=# ¥! tail -6 data/log/postgresql.log
LOG:  duration: 341.638 ms  plan:
      Query Text: SELECT COUNT(*) FROM data1;
      Aggregate  (cost=179055.00..179055.01 rows=1 width=8) (actual time=...
        -> Seq Scan on data1  (cost=0.00..154055.00 rows=10000000 width=...
          Prefetch: avg=46.37 max=91 capacity=94
          I/O: count=2397 waits=11 size=15.89 in-progress=4.27
```

3.5.2. basic_archive

起動時にパラメーターbasic_archive.archive_directory に指定されたディレクトリの存在をチェックしなくなりました。この変更は PostgreSQL 15 以降にバックポートされます。
[cc774c5]

3.5.3. file_fdw

データファイルのヘッダー・レコード数を示す header オプションが追加されました。
[26cb14a]

例 128 ヘッダーが2行の CSV ファイル読み込み

```
postgres=# ¥! cat /tmp/multi1.csv
HEADER1-1,HEADER1-2
HEADER2-1,HEADER2-2
100,data1
200,data2
postgres=# CREATE EXTENSION file_fdw ;
CREATE EXTENSION
postgres=# CREATE SERVER filesvr1 FOREIGN DATA WRAPPER file_fdw ;
CREATE SERVER
postgres=# CREATE FOREIGN TABLE multi1(col1 INT, col2 VARCHAR(10)) SERVER
    filesvr1 OPTIONS (filename '/tmp/multi1.csv', HEADER '2', format 'csv') ;
CREATE FOREIGN TABLE
postgres=# SELECT * FROM multi1 ;
 col1 | col2
-----+-----
  100 | data1
  200 | data2
(2 rows)
```

3.5.4. injection_points

injection_points モジュールには以下の新機能が追加されました。

□ injection_points_attach 関数

injection_points_attach 関数に引数が追加されたバージョンが追加されました。

[\[16a2f70\]](#)

□ injection_points_list 関数

インジェクション・ポイントの一覧を返す injection_points_list 関数が追加されました。

[\[4eca711\]](#)

□ 関数の削除

統計情報に関する以下の関数が削除されました。 [\[d52c24b\]](#)

- injection_points_stats_numcalls
- injection_points_stats_drop
- injection_points_stats_fixed

3.5.5. pg_buffercache

pg_buffercache モジュールには以下の新機能が実装されました。

□ ビューの追加

以下のビューが追加されました。[4b203d4]

表 37 追加されたビュー

ビュー名	説明
pg_buffercache_os_pages	バッファプールと OS ページのマッピング
pg_buffercache_numa	バッファプールと NUMA ノードのマッピング

□ 関数の追加

以下の関数が追加されました。[9ccc049, 4b203d4]

表 38 追加された関数

関数名	説明
pg_buffercache_os_pages	バッファプールと OS ページのマッピングを提供
pg_buffercache_mark_dirty	指定されたバッファをダーティ状態に変更する
pg_buffercache_mark_dirty_relation	バッファプール内の指定されたリレーションのバッファをダーティ状態に変更する
pg_buffercache_mark_dirty_all	バッファプール内の全バッファをダーティ状態に変更する

例 129 pg_buffercache_mark_dirty_all 関数の実行

```
postgres=# SELECT * FROM pg_buffercache_mark_dirty_all();
-[ RECORD 1 ]-----+-----
buffers_dirtied      | 5527
buffers_already_dirty | 10857
buffers_skipped       | 0
```

3.5.6. pg_plan_advice

PostgreSQL 19 に新しい Contrib モジュール `pg_plan_advice` が追加されました。このモジュールは実行計画の変更と変更された実行計画を確認する機能を提供します。[[5883ff3](#), [6455e55](#), [26255a3](#)]

□ 基本的な使い方

このモジュールは以下の手順で使用します。

1. `LOAD` 文または `shared_preload_libraries` パラメーターに `pg_plan_advice` を指定します。
2. `PLAN_ADVICE` オプションを指定した `EXPLAIN` 文により実行計画を確認します。
3. 実行計画を変更するための情報が「Generated Plan Advice:」項目に出力されます。
4. パラメーター `pg_plan_advice.advice` に実行計画の変更に関する情報を指定します。
5. 実行計画が変更されます。

以下の例ではテーブル `data1` と `data2` の結合順序を変更して実行計画が変わることを確認しています。

例 130 実行計画変更方法の確認

```
postgres=# LOAD 'pg_plan_advice' ;
LOAD
postgres=# EXPLAIN (COSTS OFF, PLAN_ADVICE) SELECT * FROM data1 AS d1
           INNER JOIN data2 AS d2 ON d1.col1 = d2.col1 WHERE d1.col1 < 1000 ;
           QUERY PLAN
-----
Nested Loop
  -> Index Scan using data1_pkey on data1 d1
      Index Cond: (col1 < 1000)
  -> Index Scan using data2_pkey on data2 d2
      Index Cond: (col1 = d1.col1)
Generated Plan Advice:
  JOIN_ORDER(d1 d2)
  NESTED_LOOP_PLAIN(d2)
  INDEX_SCAN(d1 public.data1_pkey d2 public.data2_pkey)
  NO_GATHER(d1 d2)
(10 rows)
```

例 131 実行計画変更の設定

```
postgres=# SET pg_plan_advice.advice = 'JOIN_ORDER(d2 d1)' ;
SET
postgres=# EXPLAIN (COSTS OFF) SELECT * FROM data1 AS d1 INNER JOIN data2 AS
           d2 ON d1.col1 = d2.col1 WHERE d1.col1 < 1000 ;
           QUERY PLAN
-----
Gather
  Workers Planned: 2
    -> Hash Join
          Hash Cond: (d2.col1 = d1.col1)
            -> Parallel Seq Scan on data2 d2
            -> Hash
                  -> Index Scan using data1_pkey on data1 d1
                      Index Cond: (col1 < 1000)

Supplied Plan Advice:
  JOIN_ORDER(d2 d1) /* matched */
(10 rows)
```

実行計画を変更するための情報はパラメーター`pg_plan_advice.advice`に指定します。主要な設定を以下に示します。

表 39 代表的な `advice` パラメーター設定値

設定値	説明
<code>SEQ_SCAN(target)</code>	順次検索 (Seq Scan) を使用する
<code>INDEX_SCAN(target)</code>	指定されたインデックスを使用する
<code>JOIN_ORDER(item item...)</code>	結合順序を指定する
<code>HASH_JOIN(target)</code>	ハッシュ結合を行う
<code>NESTED_LOOP_PLAN(target)</code>	ネステッド・ループ結合を行う
<code>GATHER(item)</code>	パラレルクエリーの指定
<code>NO_GATHER(item)</code>	パラレルクエリーを使わない
<code>DO_NOT_SCAN(item)</code>	指定されたリレーシヨンのスキャンを停止する

□ パラメーター

pg_plan_advice モジュールには以下の独自パラメーターが提供されています。

表 40 追加されたパラメーター

パラメーター名	説明	デフォルト値
advice	実行計画を変更するアドバイス	"
always_explain_supplied_adv ice	実行計画を変更した子を示す出力を EXPLAIN 文の実行結果に常に出力	on
always_store_advice_details	プリペアド文に対してフィードバッ クを出力するか	off
feedback_warnings	アドバイスが正常に適用されなかつ た場合に警告を出力するか	off
trace_mask	デバッグ用トレースを出力するか	off

always_explain_supplied_advice パラメーターを on (デフォルト値) に設定すると、実行計画が変更された場合に EXPLAIN 文にその情報 (Supplied Plan Advice:) が常に出力されます。このパラメーターを off に設定している場合に実行計画の変更情報を出力するためには EXPLAIN 文に PLAN_ADVICE オプションを指定する必要があります。

3.5.7. pg_stash_advice

PostgreSQL 19 に新しい Contrib モジュール pg_stash_advice が追加されました。pg_stash_advice モジュールはクエリーID に対して実行計画のアドバイスを適用します。デフォルトでは生成されたアドバイスはメモリー上に保存され、永続化されません。永続化する場合はパラメーターpg_stash_advice.persist を true に設定します。永続化データはデータベース・クラスタにファイル pg_stash_advice.tsv として作成されます。[[e8ec19a](#), [c10edb1](#)]

表 41 追加される関数

関数名	説明	備考
pg_create_advice_stash	Advice Stash の作成	
pg_drop_advice_stash	Advice Stash の削除	
pg_set_stashed_advice	クエリーにアドバイスを設定	
pg_get_advice_stashes	Advice Stash の一覧	
pg_get_advice_stash_contents	Advice Stash の内容確認	

このモジュールには以下のパラメーターが定義されています。

表 42 追加されるパラメーター

パラメーター名	説明	デフォルト値
pg_stash_advice.persist	アドバイスを永続化するか	false
pg_stash_advice.stash_name	アクティブなアドバイス集の名前	"
pg_stash_advice.persist_interval	永続化のチェック間隔 (秒)	30

□ 準備

パラメーターshared_preload_libraries に pg_stash_advice を設定します。パラメーター compute_query_id は auto (デフォルト) または on に設定します。次に CREATE EXTENSION 文でモジュールを有効化します。

例 132 pg_stash_advice モジュールの準備

```
postgres=# SHOW shared_preload_libraries ;
shared_preload_libraries
-----
pg_stash_advice
(1 row)

postgres=# SHOW compute_query_id ;
compute_query_id
-----
on
(1 row)

postgres=# CREATE EXTENSION pg_stash_advice ;
CREATE EXTENSION
```

アドバイスを集を作成します。pg_create_advice_stash 関数に名前を指定し、作成したアドバイスを pg_advice_stash.stash_name パラメーターに設定します。

例 133 アドバイス集の作成と設定

```
postgres=# SELECT pg_create_advice_stash('stash1') ;
pg_create_advice_stash
-----

(1 row)

postgres=# SET pg_stash_advice.stash_name = 'stash1' ;
SET
```

実行計画を変更したいクエリーのクエリーID と実行計画を確認します。

例 134 クエリーID の確認

```
postgres=# EXPLAIN (VERBOSE, COSTS OFF) SELECT * FROM data1 AS d1 INNER
JOIN data2 AS d2 ON d1.col1 = d2.col1 WHERE d1.col1 = 10000 ;
QUERY PLAN
-----

Nested Loop
Output: d1.col1, d1.col2, d2.col1, d2.col2
-> Index Scan using data1_pkey on public.data1 d1
    Output: d1.col1, d1.col2
    Index Cond: (d1.col1 = 10000)
-> Index Scan using data2_pkey on public.data2 d2
    Output: d2.col1, d2.col2
    Index Cond: (d2.col1 = 10000)
Query Identifier: 1524396781679940627
(9 rows)
```

クエリーID に対して実行計画を変更するアドバイスを指定します。以下の例では結合順序を変更しています。実行計画を変更する指示は `pg_plan_advice` Contrib モジュールを参照してください。

例 135 アドバイスの指定

```
postgres=# SELECT pg_set_stashed_advice('stash1', 1524396781679940627,  
      'JOIN_ORDER(d2 d1)') ;  
pg_set_stashed_advice  
-----  
  
(1 row)
```

実行計画を確認します。Supplied Plan Advice 項目が出力され、実行計画の変更アドバイスが指定されたことがわかります。

例 136 実行計画の確認

```
postgres=# EXPLAIN (VERBOSE, COSTS OFF) SELECT * FROM data1 AS d1 INNER JOIN  
data2 AS d2 ON d1.col1 = d2.col1 WHERE d1.col1 = 10000 ;  
QUERY PLAN  
-----  
Nested Loop  
  Output: d1.col1, d1.col2, d2.col1, d2.col2  
  -> Index Scan using data2_pkey on public.data2 d2  
      Output: d2.col1, d2.col2  
      Index Cond: (d2.col1 = 10000)  
  -> Index Scan using data1_pkey on public.data1 d1  
      Output: d1.col1, d1.col2  
      Index Cond: (d1.col1 = 10000)  
Query Identifier: 1524396781679940627  
Supplied Plan Advice:  
  JOIN_ORDER(d2 d1) /* matched */  
(11 rows)
```

3.5.8. pg_stat_statements

pg_stat_statements モジュールには以下の新機能が追加されました。

□ pg_stat_statements ビューの変更

pg_stat_statements ビューには以下の列が追加されました。[3357471]

表 43 追加された列

列名	データ型	説明
generic_plan_calls	bigint	汎用実行計画の実行回数
custom_plan_calls	bigint	カスタム実行計画の実行回数

例 137 実行計画実行回数

```
postgres=> SELECT query, generic_plan_calls, custom_plan_calls FROM
              pg_stat_statements WHERE query LIKE 'PREPARE%' ;
-[ RECORD 1 ]-----+-----
query              | PREPARE prepare1(int) AS SELECT $1
generic_plan_calls | 4
custom_plan_calls  | 4
```

□ FETCH 文の定数化

FETCH 文に指定された取得数の数字は定数化され単一のクエリーID が付与されるようになりました。[bee23ea]

例 138 FETCH 文の定数化

```
postgres=> BEGIN ;
BEGIN
postgres=> DECLARE c1 CURSOR FOR SELECT * FROM data1 ;
DECLARE CURSOR
postgres=> FETCH 1 FROM c1 ;
  c1 |  c2
-----+-----
   1 | data1
(1 row)
postgres=> FETCH 2 FROM c1 ;
  c1 |  c2
-----+-----
   2 | data2
   3 | data3
(2 rows)
postgres=> SELECT query, calls FROM pg_stat_statements WHERE query LIKE
'FETCH%' ;
      query      | calls
-----+-----
FETCH $1 FROM c1 |      2
(1 row)
```

3.5.9. postgres_fdw

postgres_fdw モジュールには以下の機能が追加されました。

□ restore_stats オプション

FOREIGN DATA WRAPPER にリモートの統計情報をインポートするコールバックが追加されました。postgres_fdw モジュールはこのコールバックを実装し、restore_stats オプションとして FOREIGN SERVER と外部テーブルに指定できます。[[28972b6](#)]

例 139 `restore_stats` オプションの指定

```
postgres=> CREATE FOREIGN TABLE data1(col1 INT, col2 VARCHAR(10)) SERVER dbsvr1
           OPTIONS (restore_stats 'true') ;

CREATE FOREIGN TABLE
postgres=> ANALYZE (VERBOSE) data1 ;
INFO:  importing statistics for foreign table "public.data1"
INFO:  finished importing statistics for foreign table "public.data1"
ANALYZE
```

□ ARRAY 操作のプッシュダウン

汎用プラン使用時にプリペア文の `ANY` 句がプッシュダウンできるようになりました。

[\[62c3b4c\]](#)

例 140 実行する SQL 文

```
postgres=> SET plan_cache_mode = force_generic_plan ;
SET
postgres=> PREPARE array_stmt(VARCHAR[]) AS SELECT * FROM data1 WHERE
           col2 = ANY($1) ;

PREPARE
postgres=> EXPLAIN (VERBOSE, COSTS OFF) EXECUTE
           array_stmt(ARRAY['data1', 'data2']) ;
```

例 141 PostgreSQL 18 の実行計画

```
postgres=> EXPLAIN (VERBOSE, COSTS OFF) EXECUTE
           array_stmt(ARRAY['data1', 'data2']) ;
           QUERY PLAN

-----
Foreign Scan on public.data1
  Output: col1, col2
  Filter: ((data1.col2)::text = ANY (($1)::text[]))
  Remote SQL: SELECT col1, col2 FROM public.data1
(4 rows)
```

例 142 PostgreSQL 19 の実行計画

```
postgres=> EXPLAIN (VERBOSE, COSTS OFF) EXECUTE
           array_stmt(ARRAY['data1', 'data2']) ;
           QUERY PLAN
-----
Foreign Scan on public.data1
  Output: col1, col2
  Remote SQL: SELECT col1, col2 FROM public.data1 WHERE ((col2 = ANY
($1::character varying[])))
(3 rows)
```

□ トランザクション状態の伝播
ローカルの **READ ONLY** トランザクション、**DEFERRABLE** トランザクションの情報がリモート接続にも伝播するようになりました。[[de28140](#)]

3.5.10. その他

上記で説明した以外に、以下のテスト用拡張モジュールが `src/test/modules` ディレクトリ以下に追加されました。[[31280d9](#), [e0e4c13](#)]

表 44 追加された拡張モジュール

モジュール名	説明
test_binaryheap	バイナリヒープのテスト
test_bitmapset	Bitmapset のテスト
test_checksums	チェックサム機能の変更テスト
test_cplusplusext	ヘッダーの C++コンパイラーのコンパイル・テスト
test_custom_stats	固定・可変サイズのカスタム統計情報のテスト
test_custom_types	カスタム・タイプのテスト
test_int128	128 ビット int 型のテスト
test_lwlock_tranches	LWLock Tranche のテスト
test_plan_advice	pg_plan_advice モジュールのテスト
test_saslprep	SASLprep の実装テスト
test_shmem	共有メモリーのテスト

参考にした URL

本資料の作成には、以下の URL を参考にしました。

- PostgreSQL 19 Release Notes
<https://www.postgresql.org/docs/19/release.html>
- PostgreSQL 19 Manual
<https://www.postgresql.org/docs/19/index.html>
- PostgreSQL 19 Beta 1 のアナウンス
<https://www.postgresql.org/about/news/postgresql-19-beta-1-released-3313/>
- PostgreSQL 19 Open Items
https://wiki.postgresql.org/wiki/PostgreSQL_19_Open_Items
- pgPedia
<https://pgpedia.info/postgresql-versions/postgresql-19.html>
- Commitfests
<https://commitfest.postgresql.org/>
- Git
`git://git.postgresql.org/git/postgresql.git`
- GitHub
<https://github.com/postgres/postgres>
- pgsql-hackers Mailing list
<https://www.postgresql.org/list/pgsql-hackers/>
- PostgreSQL Developer Information
https://wiki.postgresql.org/wiki/Development_information
- Slack - postgresql-jp (Japanese)
<https://postgresql-jp.slack.com/>

変更履歴

変更履歴

版	日付	作成者	説明
1.0	2026/06/04	篠田典良	PostgreSQL 19 Beta 1 公開版を作成

以上

